

SOLIDWORKS DimXpert API (.NET) — Examples (1/7): Auto Dimension Scheme Example CSharp- Get_and_Set_Location_Dimension_Example_V

SOLIDWORKS API Documentation (converted from CHM)

Auto Dimension Scheme Example (C#)

This example shows how to create a DimXpert Auto Dimension Scheme, turn tolerance status on and off, and delete tolerances.

```
//-----
//
// Preconditions:
// 1. Open public_documents\samples\tutorial\dimxpert\bracket_auto_plusminus.sldprt.
// 2. (Optional) Multi-select three faces to specify primary, secondary, and tertiary datum.
// 3. Open the Immediate window.
// 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:
//    a. Right-click the project in Project Explorer.
//    b. Select Add Reference.
//    c. Click the Browse tab.
//    d. Find and select install_dir\api\redist\swdimxpert.dll.
// 5. Set two breakpoints:
//    * swDXPart.ShowToleranceStatus = false;
//    * retval = swDXPart.DeleteAllTolerances();
// 6. Press F5.
// 7. Observe on the DimXpertManager tab: Hole Pattern1, Hole Pattern2, Fillet1,
//    Fillet Pattern1. Also observe that tolerance status is turned on in
//    the SOLIDWORKS viewer.
// 8. Press F5 and observe that tolerance status is turned off.
// 9. Compare the output in the Immediate window with the features displayed
//    on the DimXpertManager tab.
// 10. Press F5 and observe that all tolerance annotations have been removed from the part.
//
// Postconditions: None
//
// NOTE: Because this part is used elsewhere, do not save changes.
//-----
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdimxpert;
using System;
using System.Diagnostics;
namespace AutoDimScheme_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        SelectionMgr swSelMgr;
        Feature swFeature;
        DimXpertManager swSchema;
        DimXpertPart swDXPart;
        DimXpertAutoDimSchemeOption schemeOption;
        swDimXpertFeatureType_e featureType;
        object[] features;
        object[] appliedFeatures;
        object[] appliedAnnotations;
        DimXpertAnnotation appliedAnnotation;
        DimXpertFeature feature;
        DimXpertFeature appliedFeature;
        string msgStr;
        string msgStr2;
        string msgStr3;
        string msgStr4;
        int n;
        int o;
        int p;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
            swSchema = swModelDocExt.get_DimXpertManager("Default", true);

            // Get IDimXpertPart from the IDimXpertManager
            swDXPart = (DimXpertPart)swSchema.DimXpertPart;

            // Set Auto Dimension Scheme using default options
            schemeOption = swDXPart.GetAutoDimSchemeOption();
            Debug.Print("Default for ScopeAllFeature is: ");
            Debug.Print(schemeOption.ScopeAllFeature.ToString());
            Debug.Print("Default for FeatureFilters is: ");
            Debug.Print(schemeOption.FeatureFilters.ToString());
            Debug.Print("Default for PartType is: ");
            Debug.Print(schemeOption.PartType.ToString());
            Debug.Print("Default for PatternType is: ");
            Debug.Print(schemeOption.PatternType.ToString());
            Debug.Print("Default for PolarPatternHoleCount is: ");
```

```

Debug.Print(schemeOption.PolarPatternHoleCount.ToString());
Debug.Print("Default for ToleranceType is: ");
Debug.Print(schemeOption.ToleranceType.ToString());

bool retval = false;
retval = swDXPart.AutoDimensionScheme(schemeOption);

// Turn tolerance status off
swDXPart.ShowToleranceStatus = false;

long featCount = 0;
featCount = swDXPart.GetFeatureCount();

msgStr = "Total of ";
msgStr2 = featCount.ToString();
msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName);

Debug.Print("");
Debug.Print(msgStr);

// Get IDimXpert features through IDimXpertPart
features = (object[])swDXPart.GetFeatures();
msgStr = (swSchema.SchemaName) + " has the following features: ";

Debug.Print("");
Debug.Print(msgStr);

for (n = 0; n <= features.GetUpperBound(0); n++)
{
    //Use IDimXpertFeature to get feature data
    feature = (DimXpertFeature)features[n];

    Debug.Print(" " + "Feature name: " + (feature.Name));

    featureType = feature.Type;
    GetPatternType(ref featureType, ref msgStr2);

    msgStr = " Feature type ";
    msgStr3 = " is suppressed on the DimXpertManager tab? ";
    msgStr4 = feature.IsSuppressed().ToString();

    Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4);

    msgStr = " " + "Swift model: ";

    //Use IFeature to get the Swift model corresponding to this geometric dimensioning and tolerancing data
    swFeature = (Feature)feature.GetModelFeature();

    if ((swFeature != null))
    {
        msgStr2 = swFeature.GetTypeName2();
        Debug.Print(msgStr + msgStr2);
    }

    msgStr = " " + "Number of SOLIDWORKS face entities in this feature: ";
    msgStr2 = feature.GetFaceCount().ToString();

    Debug.Print(msgStr + msgStr2);

    msgStr = " " + "Number of applied features: ";
    msgStr2 = feature.GetAppliedFeatureCount().ToString();

    Debug.Print(msgStr + msgStr2);

    appliedFeatures = (object[])feature.GetAppliedFeatures();

    if (!((appliedFeatures == null)))
    {
        for (o = 0; o <= appliedFeatures.GetUpperBound(0); o++)
        {
            appliedFeature = (DimXpertFeature)appliedFeatures[o];
            Debug.Print(" " + "Applied feature name: " + (appliedFeature.Name));
        }
    }

    msgStr = " " + "Number of applied annotations: ";
    msgStr2 = feature.GetAppliedAnnotationCount().ToString();
    Debug.Print(msgStr + msgStr2);

    appliedAnnotations = (object[])feature.GetAppliedAnnotations();

    if (!((appliedAnnotations == null)))
    {
        for (p = 0; p <= appliedAnnotations.GetUpperBound(0); p++)
        {
            appliedAnnotation = (DimXpertAnnotation)appliedAnnotations[p];
            Debug.Print(" " + "Applied annotation name: " + (appliedAnnotation.Name));
        }
    }
    Debug.Print(" ");
}

// Delete all tolerances
retval = swDXPart.DeleteAllTolerances();
}

```

```

public void GetPatternType(ref swDimXpertFeatureType_e featureType, ref String msgStr2)
{
    if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Plane))
    {
        msgStr2 = "Plane";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Cylinder))
    {
        msgStr2 = "Cylinder";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Cone))
    {
        msgStr2 = "Cone";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Extrude))
    {
        msgStr2 = "Extrude";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Fillet))
    {
        msgStr2 = "Fillet";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Chamfer))
    {
        msgStr2 = "Chamfer";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole))
    {
        msgStr2 = "CompoundHole";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth))
    {
        msgStr2 = "CompoundWidth";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch))
    {
        msgStr2 = "CompoundNotch";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D))
    {
        msgStr2 = "CompoundClosedSlot3D";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint))
    {
        msgStr2 = "IntersectPoint";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine))
    {
        msgStr2 = "IntersectLine";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle))
    {
        msgStr2 = "IntersectCircle";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane))
    {
        msgStr2 = "IntersectPlane";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Pattern))
    {
        msgStr2 = "Pattern";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Sphere))
    {
        msgStr2 = "Sphere";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane))
    {
        msgStr2 = "Bestfit plane";
    }
    else if ((featureType == swDimXpertFeatureType_e.swDimXpertFeature_Surface))
    {
        msgStr2 = "Surface";
    }
}

public SldWorks swApp;
}
}

```

Auto Dimension Scheme Example (VBA)

This example shows how to create a DimXpert Auto Dimension Scheme, turn tolerance status on and off, and delete tolerances.

```

'-----
' Preconditions:
'
' 1. Open public_documents\samples\tutorial\dimxpert\bracket_auto_plusminus.sldprt.
' 2. (Optional) Multi-select three faces to specify primary, secondary,
'    and tertiary datum.
' 3. Open an Immediate window.
' 4. Ensure that the latest SOLIDWORKS DimXpert type library is loaded:
'    a. Click Tools > References.
'    b. Click Browse.
'    c. Find and select install_dir\swdimxpert.tlb.
' 5. Set two breakpoints on these lines:
'    * swDXPart.ShowToleranceStatus = False
'    * retval = swDXPart.DeleteAllTolerances
' 6. Press F5.
' 7. Observe on the DimXpertManager tab: Hole Pattern1, Hole Pattern2,
'    Fillet1, Fillet Pattern1. Also observe that tolerance status is turned
'    on in the SOLIDWORKS viewer.
' 8. Press F5 and observe that tolerance status is turned off.
' 9. Compare the output in the Immediate window with the features
'    displayed on the DimXpertManager tab.
' 10. Press F5 and observe that all tolerance annotations have been
'     removed from the model.
'
' Postconditions: None
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swConfig As SldWorks.Configuration
Dim swFeature As SldWorks.feature
Dim swAnn As SldWorks.feature
Dim swSchema As SldWorks.DimXpertManager
Dim swDXPart As DimXpertPart
Dim schemeOption As DimXpertAutoDimSchemeOption
Dim featureType As swDimXpertFeatureType_e
Dim features As Variant
Dim appliedFeatures As Variant
Dim appliedAnnotations As Variant
Dim appliedAnnotation As DimXpertAnnotation
Dim feature As DimXpertFeature
Dim appliedFeature As DimXpertFeature
Dim msgStr As String
Dim msgStr2 As String
Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long

Dim boolstatus As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    Set swSchema = swModelDocExt.DimXpertManager("Default", True)

    ' Get IDimXpertPart from the IDimXpertManager
    Set swDXPart = swSchema.DimXpertPart

    ' Set Auto Dimension Scheme using default options
    Set schemeOption = swDXPart.GetAutoDimSchemeOption
    Debug.Print "Default for ScopeAllFeature is: "
    Debug.Print (schemeOption.ScopeAllFeature)
    Debug.Print "Default for FeatureFilters is: "
    Debug.Print (schemeOption.FeatureFilters)
    Debug.Print "Default for PartType is: "
    Debug.Print (schemeOption.PartType)
    Debug.Print "Default for PatternType is: "
    Debug.Print (schemeOption.PatternType)
    Debug.Print "Default for PolarPatternHoleCount is: "
    Debug.Print (schemeOption.PolarPatternHoleCount)
    Debug.Print "Default for ToleranceType is: "
    Debug.Print (schemeOption.ToleranceType)

    Dim retval As Boolean
    retval = swDXPart.AutoDimensionScheme(schemeOption)

```

```

' Turn tolerance status off
swDXPart.ShowToleranceStatus = False

Dim featCount As Long
featCount = swDXPart.GetFeatureCount

msgStr = "Total of "
msgStr2 = featCount
msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)

Debug.Print ""
Debug.Print msgStr
' Get IDimXpert features through IDimXpertPart
features = swDXPart.GetFeatures
msgStr = (swSchema.SchemaName) + " has the following features: "

Debug.Print ""
Debug.Print msgStr

For n = 0 To UBound(features)
    'Use IDimXpertFeature to get feature data
    Set feature = features(n)

    Debug.Print " " + "Feature name: " + (feature.Name)

    featureType = feature.Type
    Call GetPatternType(featureType, msgStr2)

    msgStr = " " + "Feature type "
    msgStr3 = " is suppressed on the DimXpertManager tab? "
    msgStr4 = feature.IsSuppressed()

    Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4
    msgStr = " " + "Swift model: "

    'Use IFeature to get the Swift model corresponding to this geometric dimensioning and tolerancing data
    Set swFeature = feature.GetModelFeature()

    If Not (swFeature Is Nothing) Then
        msgStr2 = swFeature.GetTypeName2()
        Debug.Print msgStr + msgStr2
    End If

    msgStr = " " + "Number of SOLIDWORKS face entities in this feature: "
    msgStr2 = feature.GetFaceCount

    Debug.Print msgStr + msgStr2

    msgStr = " " + "Number of applied features: "
    msgStr2 = feature.GetAppliedFeatureCount()

    Debug.Print msgStr + msgStr2

    appliedFeatures = feature.GetAppliedFeatures()

    If Not (IsEmpty(appliedFeatures)) Then
        For o = 0 To UBound(appliedFeatures)
            Set appliedFeature = appliedFeatures(o)
            Debug.Print " " + "Applied feature name: " + (appliedFeature.Name)
        Next
    End If

    msgStr = " " + "Number of applied annotations: "
    msgStr2 = feature.GetAppliedAnnotationCount()
    Debug.Print msgStr + msgStr2

    appliedAnnotations = feature.GetAppliedAnnotations()

    If Not (IsEmpty(appliedAnnotations)) Then
        For p = 0 To UBound(appliedAnnotations)
            Set appliedAnnotation = appliedAnnotations(p)
            Debug.Print " " + "Applied annotation name: " + (appliedAnnotation.Name)
        Next
    End If

    Debug.Print " "

Next

' Delete all tolerances
retval = swDXPart.DeleteAllTolerances

End Sub

Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    End If
End Sub

```

```
ElseIf (featureType = swDimXpertFeature_Chamfer) Then
    msgStr2 = "Chamfer"
ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
    msgStr2 = "CompoundHole"
ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
    msgStr2 = "CompoundWidth"
ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
    msgStr2 = "CompoundNotch"
ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
    msgStr2 = "CompoundClosedSlot3D"
ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
    msgStr2 = "IntersectPoint"
ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
    msgStr2 = "IntersectLine"
ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
    msgStr2 = "IntersectCircle"
ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"
ElseIf (featureType = swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"
ElseIf (featureType = swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"
ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit plane"
ElseIf (featureType = swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If
End Sub
```

Auto Dimension Scheme Example (VB.NET)

This example shows how to create a DimXpert Auto Dimension Scheme, turn tolerance status on and off, and delete tolerances.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\dimxpert\bracket_auto_plusminus.sldprt.
' 2. (Optional) Multi-select three faces to specify primary, secondary,
'    and tertiary datum.
' 3. Open the Immediate window.
' 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:
'    a. Right-click the project in Project Explorer.
'    b. Select Add Reference.
'    c. Click the Browse tab.
'    d. Find and select install_dir\api\redist\swdimxpert.dll.
' 5. Set two breakpoints on these lines:
'    * swDXPart.ShowToleranceStatus = False
'    * retVal = swDXPart.DeleteAllTolerances
' 6. Press F5.
' 7. Observe on the DimXpertManager tab: Hole Pattern1, Hole Pattern2,
'    Fillet1, Fillet Pattern1. Also observe that tolerance status is
'    turned on in the SOLIDWORKS viewer.
' 8. Press F5 and observe that tolerance status is turned off.
' 9. Compare the output in the Immediate window with the features displayed
'    on the DimXpertManager tab.
' 10. Press F5 and observe that all tolerance annotations have been
'     removed from the part.
'
' Postconditions: None
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdimxpert
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim swConfig As Configuration
    Dim swFeature As Feature
    Dim swAnn As Feature
    Dim swSchema As DimXpertManager
    Dim swDXPart As DimXpertPart
    Dim schemeOption As DimXpertAutoDimSchemeOption
    Dim featureType As swDimXpertFeatureType_e
    Dim features As Object
    Dim appliedFeatures As Object
    Dim appliedAnnotations As Object
    Dim appliedAnnotation As DimXpertAnnotation
    Dim feature As DimXpertFeature
    Dim appliedFeature As DimXpertFeature
    Dim msgStr As String
    Dim msgStr2 As String
    Dim msgStr3 As String
    Dim msgStr4 As String
    Dim n As Long
    Dim o As Long
    Dim p As Long

    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
        swSchema = swModelDocExt.DimXpertManager("Default", True)

        ' Get IDimXpertPart from the IDimXpertManager
        swDXPart = swSchema.DimXpertPart

        ' Set Auto Dimension Scheme using default options
        schemeOption = swDXPart.GetAutoDimSchemeOption
        Debug.Print("Default for ScopeAllFeature is: ")
        Debug.Print(schemeOption.ScopeAllFeature)
        Debug.Print("Default for FeatureFilters is: ")
        Debug.Print(schemeOption.FeatureFilters)
        Debug.Print("Default for PartType is: ")
        Debug.Print(schemeOption.PartType)
        Debug.Print("Default for PatternType is: ")

```

```

Debug.Print(schemeOption.PatternType)
Debug.Print("Default for PolarPatternHoleCount is: ")
Debug.Print(schemeOption.PolarPatternHoleCount)
Debug.Print("Default for ToleranceType is: ")
Debug.Print(schemeOption.ToleranceType)

Dim retVal As Boolean
retVal = swDXPart.AutoDimensionScheme(schemeOption)

' Turn tolerance status off
swDXPart.ShowToleranceStatus = False

Dim featCount As Long
featCount = swDXPart.GetFeatureCount

msgStr = "Total of "
msgStr2 = featCount
msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)

Debug.Print("")
Debug.Print(msgStr)

' Get IDimXpert features through IDimXpertPart
features = swDXPart.GetFeatures
msgStr = (swSchema.SchemaName) + " has the following features: "

Debug.Print("")
Debug.Print(msgStr)

For n = 0 To UBound(features)

    'Use IDimXpertFeature to get feature data
    feature = features(n)

    Debug.Print(" " + "Feature name: " + (feature.Name))

    featureType = feature.Type
    Call GetPatternType(featureType, msgStr2)

    msgStr = " " + "Feature type "
    msgStr3 = " is suppressed on the DimXpertManager tab? "
    msgStr4 = feature.IsSuppressed()

    Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)

    msgStr = " " + "Swift model: "

    'Use IFeature to get the Swift model corresponding to this geometric dimensioning and tolerancing data
    swFeature = feature.GetModelFeature()

    If Not (swFeature Is Nothing) Then
        msgStr2 = swFeature.GetTypeName2()
        Debug.Print(msgStr + msgStr2)
    End If

    msgStr = " " + "Number of SOLIDWORKS face entities in this feature: "
    msgStr2 = feature.GetFaceCount

    Debug.Print(msgStr + msgStr2)

    msgStr = " " + "Number of applied features: "
    msgStr2 = feature.GetAppliedFeatureCount()

    Debug.Print(msgStr + msgStr2)

    appliedFeatures = feature.GetAppliedFeatures()

    If Not (IsNothing(appliedFeatures)) Then
        For o = 0 To UBound(appliedFeatures)

            appliedFeature = appliedFeatures(o)
            Debug.Print(" " + "Applied feature name: " + (appliedFeature.Name))

        Next
    End If

    msgStr = " " + "Number of applied annotations: "
    msgStr2 = feature.GetAppliedAnnotationCount()
    Debug.Print(msgStr + msgStr2)

    appliedAnnotations = feature.GetAppliedAnnotations()

    If Not (IsNothing(appliedAnnotations)) Then
        For p = 0 To UBound(appliedAnnotations)

            appliedAnnotation = appliedAnnotations(p)
            Debug.Print(" " + "Applied annotation name: " + (appliedAnnotation.Name))

        Next
    End If

    Debug.Print(" ")

```

Next

```
' Delete all tolerances  
retval = swDXPart.DeleteAllTolerances
```

End Sub

Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)

```
If (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Plane) Then  
    msgStr2 = "Plane"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cylinder) Then  
    msgStr2 = "Cylinder"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cone) Then  
    msgStr2 = "Cone"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then  
    msgStr2 = "Extrude"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then  
    msgStr2 = "Fillet"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then  
    msgStr2 = "Chamfer"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then  
    msgStr2 = "CompoundHole"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then  
    msgStr2 = "CompoundWidth"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then  
    msgStr2 = "CompoundNotch"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D) Then  
    msgStr2 = "CompoundClosedSlot3D"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then  
    msgStr2 = "IntersectPoint"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then  
    msgStr2 = "IntersectLine"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then  
    msgStr2 = "IntersectCircle"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then  
    msgStr2 = "IntersectPlane"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then  
    msgStr2 = "Pattern"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then  
    msgStr2 = "Sphere"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then  
    msgStr2 = "Bestfit plane"  
  
ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then  
    msgStr2 = "Surface"
```

End If

End Sub

Public swApp As SldWorks

End Class

DimXpert Annotation Data Module Example (C#)

```
// This module is a component of
// Get DimXpert Features and Annotations in a Model Example \(C#\).

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swdimxpert;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Collections.ObjectModel;
public class DimXpertAnnotationData
{
    // Returns a collection of Strings containing the annotations data
    private Collection<string> strs = new Collection<string>();
    public void Clear(Collection<string> strs)
    {
        strs.Remove(strs[strs.Count]);
        if (!(strs.Count == 0))
        {
            Clear(strs);
        }
    }
    public Collection<string> AnnotationData(DimXpertAnnotation annotation, SolidWorks.Interop.swdimxpert.DimXpertPart
dimXpertPart)
    {
        swDimXpertAnnotationType_e annoType = default(swDimXpertAnnotationType_e);

        if (!(strs.Count == 0))
        {
            Clear(strs);
        }
        //General info
        GeneralInfo(annotation);
        annoType = annotation.Type;
        if (annoType == swDimXpertAnnotationType_e.swDimXpertDatum)
        {
            DatumData((DimXpertDatum)annotation);
        }
        else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Position)
        {
            PositionData((DimXpertPositionTolerance)annotation);
        }
        else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition)
        {
            CompositePositionData((DimXpertCompositePositionTolerance)annotation);
        }
        else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry)
        {
            SymmetryData((DimXpertSymmetryTolerance)annotation);
        }
        else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity)
        {
            ConcentricityData((DimXpertConcentricityTolerance)annotation);
        }
        else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile)
        {
            LineProfileData((DimXpertLineProfileTolerance)annotation);
        }
    }
}
```

```

    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile)
    {
        CompositeLineProfileData((DimXpertCompositeLineProfileTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile)
    {
        SurfaceProfileData((DimXpertSurfaceProfileTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile)
    {
        CompositeSurfaceProfileData((DimXpertCompositeSurfaceProfileTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity | annoType ==
swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism | annoType ==
swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity)
    {
        OrientationData((DimXpertOrientationTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout)
    {
        TotalRunoutData((DimXpertTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout)
    {
        CircularRunoutData((DimXpertTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness)
    {
        FlatnessData((DimXpertFlatnessTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity)
    {
        CircularityData((DimXpertTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity)
    {
        CylindricityData((DimXpertTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness)
    {
        StraightnessData((DimXpertStraightnessTolerance)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency)
    {
        TangencyData((DimXpertTangencyTolerance)annotation);
    }
    // any of the dimension tolerance types
    else
    {
        DimensionToleranceData((DimXpertDimensionTolerance)annotation, (DimXpertPart) dimXpertPart);
    }
    return str;
}

private void GeneralInfo(DimXpertAnnotation annotation)
{
    string annoType = null;
    Feature modelObj = default(Feature);
    str.Add("Name: " + annotation.Name);
    annoType = annotationTypeNameFromObject(annotation);
    str.Add("Type: " + annoType);
    str.Add("Display Entity: " + DisplayEntity(annotation));
}

```

```

        str.Add("Is Suppressed: " + (annotation.IsSuppressed() ? "True" : "False"));
        str.Add("Inspection Dimension: " + (annotation.IsToBeInspected() ? "True" : "False"));
        str.Add("Is Statistical: " + (annotation.IsStatistical() ? "True" : "False"));
        str.Add("Is FreeState: " + (annotation.IsFreeState() ? "True" : "False"));
        modelObj = (Feature)annotation.GetModelFeature();
        if ((modelObj != null))
        {
            str.Add("ModelFeature: " + modelObj.Name + " (" + modelObj.GetTypeName2() + ")");
        }
    }
}

private void DatumData(DimXpertDatum annotation)
{
    // the datum letter
    str.Add("");
    str.Add("Datum Letter: " + annotation.Identifier);
}

private void PositionData(DimXpertPositionTolerance annotation)
{
    double I = 0;
    double J = 0;
    double K = 0;
    bool enabled = false;
    double value = 0;
    bool boolstatus = false;
    str.Add("");
    str.Add("Tolerance Compartment:");
    // the shape of the tolerance zone
    str.Add(" Zone Type: " + PositionZoneType((long)annotation.ZoneType));
    // the zone vector if the tolerance zone is planar
    if (annotation.ZoneType == swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition)
    {
        boolstatus = annotation.GetPlanarZoneVector(ref I, ref J, ref K);
        str.Add(" Direction: " + FormatVector(I, J, K));
    }
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    str.Add(" Max Tolerance: " + FormatDouble(tol.Tolerance));
    // the material condition modifier applied to feature
    str.Add(" Modifier: " + mcmStr((long)annotation.Modifier));
    // the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(ref enabled, ref value);
    FormatProjectedZone(enabled, value);
    // the datum reference frame
    DatumsStr((DimXpertTolerance)annotation);
}

private void CompositePositionData(DimXpertCompositePositionTolerance annotation)
{
    double I = 0;
    double J = 0;
    double K = 0;
    bool enabled = false;
    double value = 0;
    bool boolstatus = false;
    str.Add("");
    str.Add("Tolerance Compartment:");
    // the shape of the tolerance zone
    str.Add(" Zone Type: " + PositionZoneType((long)annotation.ZoneType));
    // the zone vector when the zone is planar
    if (annotation.ZoneType == swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition)
    {
        boolstatus = annotation.GetPlanarZoneVector(ref I, ref J, ref K);
    }
}

```

```

        strs.Add(" Direction: " + FormatVector(I, J, K));
    }
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the material condition modifier
    strs.Add(" Modifier: " + mcmStr((long)annotation.Modifier));
    // the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(ref enabled, ref value);
    FormatProjectedZone(enabled, value);
    // the datum reference frame for the pattern location
    DatumsStr((DimXpertTolerance)annotation);
    // the datum reference frame for the feature to feature location
    strs.Add("Composite datums:");
    strs.Add(" Repeat Primary: " + (annotation.PrimaryInLowerTier ? "True" : "False"));
    strs.Add(" Repeat Secondary: " + (annotation.SecondaryInLowerTier ? "True" : "False"));
    strs.Add(" Repeat Tertiary: " + (annotation.TertiaryInLowerTier ? "True" : "False"));
}
private void SymmetryData(DimXpertSymmetryTolerance annotation)
{
    double I = 0;
    double J = 0;
    double K = 0;
    bool boolstatus = false;
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the material condition modifier applied to feature
    strs.Add(" Modifier: " + mcmStr((long)annotation.Modifier));
    // the datum reference frame
    strs.Add("");
    DatumsStr((DimXpertTolerance)annotation);
    // the direction of the planar zone
    strs.Add("");
    boolstatus = annotation.GetZoneDirection(ref I, ref J, ref K);
    strs.Add("Planar Zone Direction: " + FormatVector(I, J, K));
}
private void ConcentricityData(DimXpertConcentricityTolerance annotation)
{
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the material condition modifier applied to feature
    strs.Add(" Modifier: " + mcmStr((long)annotation.Modifier));
    // the datum reference frame
    DatumsStr((DimXpertTolerance)annotation);
}
private void TotalRunoutData(DimXpertTolerance annotation)
{
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
}

```

```

        // the datum reference frame
        DatumsStr(annotation);
    }
private void CircularRunoutData(DimXpertTolerance annotation)
{
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the datum reference frame
    DatumsStr(annotation);
}
private void LineProfileData(DimXpertLineProfileTolerance annotation)
{
    double I = 0;
    double J = 0;
    double K = 0;
    bool boolstatus = false;
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the outer (outside material) tolerance value
    strs.Add(" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue));
    // the vector normal to the profile zones
    strs.Add("");
    boolstatus = annotation.GetPlanarZoneVector(ref I, ref J, ref K);
    strs.Add("Planar Zone Vector: " + FormatVector(I, J, K));
    // the datum reference frame
    DatumsStr((DimXpertTolerance)annotation);
}
private void CompositeLineProfileData(DimXpertCompositeLineProfileTolerance annotation)
{
    bool boolstatus = false;
    double i = 0;
    double j = 0;
    double k = 0;
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the outer (outside material) tolerance value
    strs.Add(" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue));
    // the vector normal to the profile zones
    strs.Add("");
    boolstatus = annotation.GetPlanarZoneVector(ref i, ref j, ref k);
    strs.Add("Planar Zone Vector: " + FormatVector(i, j, k));
    // the datum reference frame
    DatumsStr((DimXpertTolerance)annotation);
    // the datum reference frame for the orientation and form
    strs.Add("Composite Datums:");
    strs.Add(" Repeat Primary: " + (annotation.PrimaryInLowerTier ? "True" : "False"));
    strs.Add(" Repeat Secondary: " + (annotation.SecondaryInLowerTier ? "True" : "False"));
    strs.Add(" Repeat Tertiary: " + (annotation.TertiaryInLowerTier ? "True" : "False"));
}
private void SurfaceProfileData(DimXpertSurfaceProfileTolerance annotation)

```

```

{
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the outer (outside material) tolerance value
    strs.Add(" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue));
    // the datum reference frame
    DatumsStr((DimXpertTolerance)annotation);
}

private void CompositeSurfaceProfileData(DimXpertCompositeSurfaceProfileTolerance annotation)
{
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance Upper Tier: " + FormatDouble(tol.Tolerance));
    // the outer tolerance value
    strs.Add(" Outer Tolerance Upper Tier: " + FormatDouble(annotation.OuterToleranceValue));
    // the datum reference frame for the location
    DatumsStr((DimXpertTolerance)annotation);
    // the datum reference frame for the orientation and form
    strs.Add("Composite Datums:");
    strs.Add(" Repeat Primary: " + (annotation.PrimaryInLowerTier ? "True" : "False"));
    strs.Add(" Repeat Secondary: " + (annotation.SecondaryInLowerTier ? "True" : "False"));
    strs.Add(" Repeat Tertiary: " + (annotation.TertiaryInLowerTier ? "True" : "False"));
}

private void OrientationData(DimXpertOrientationTolerance annotation)
{
    double I = 0;
    double J = 0;
    double K = 0;
    bool enabled = false;
    double value = 0;
    SolidWorks.Interop.swdimxpert.swDimXpertAnnotationType_e annoType =
default(SolidWorks.Interop.swdimxpert.swDimXpertAnnotationType_e);
    bool boolstatus = false;
    DimXpertAnnotation anno;
    anno = (DimXpertAnnotation)annotation;
    annoType = (SolidWorks.Interop.swdimxpert.swDimXpertAnnotationType_e)anno.Type;
    // the type or orientation tolerance
    if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity)
    {
        strs.Add("Orientation Type: Perpendicularity");
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism)
    {
        strs.Add("Orientation Type: Parallelism");
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity)
    {
        strs.Add("Orientation Type: Angularity");
    }
    strs.Add("");
    strs.Add("Tolerance Compartment:");
    // the shape of the tolerance zone
    switch (annotation.ZoneType)
    {
        case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Cylindrical:
            strs.Add(" Zone Type: Cylindrical");
    }
}

```

```

        break;
    case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Planar:
        str.Add(" Zone Type: Planar");
        boolstatus = annotation.GetPlanarZoneVector(ref I, ref J, ref K);
        str.Add("Planar Zone Vector: " + FormatVector(I, J, K));
        break;
    }
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    str.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    //material condition modifier applied to feature
    str.Add(" Modifier: " + mcmStr((long)annotation.Modifier));
    // the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(ref enabled, ref value);
    FormatProjectedZone(enabled, value);
    // is tangent plane
    str.Add(" IsTangentPlane: " + (annotation.IsTangentPlane() ? "True" : "False"));
    // the datum reference frame
    DatumsStr((DimXpertTolerance)annotation);
}
private void FlatnessData(DimXpertFlatnessTolerance annotation)
{
    bool enabled = false;
    double length = 0;
    double width = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    bool boolstatus = false;
    str.Add("");
    str.Add("Tolerance Compartment:");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    str.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the per unit area data
    boolstatus = annotation.GetPerUnitArea(ref enabled, ref length, ref width, ref I, ref J, ref K);
    str.Add(" Per Unit Area: " + (enabled ? "True" : "False"));
    if (enabled)
    {
        str.Add(" Per Unit Length: " + FormatDouble(length));
        str.Add(" Per Unit Width: " + FormatDouble(width));
        str.Add(" Per Unit Direction: " + FormatVector(I, J, K));
    }
}
private void CircularityData(DimXpertTolerance annotation)
{
    str.Add("");
    str.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    str.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
}
private void CylindricityData(DimXpertTolerance annotation)
{
    str.Add("");
    str.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;

```

```

        strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    }
private void StraightnessData(DimXpertStraightnessTolerance annotation)
{
    double I = 0;
    double J = 0;
    double K = 0;
    bool enabled = false;
    double dist = 0;
    bool boolstatus = false;
    //tolerance compartment info
    strs.Add("");
    strs.Add("Tolerance Compartment");
    //type or shape of the tolerance zone
    switch (annotation.ZoneType)
    {
        case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Cylindrical:
            strs.Add(" Zone Type: Cylindrical");
            break;
        case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_PlanarMedian:
            strs.Add(" Zone Type: Planar Median");
            break;
        case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Surface:
            strs.Add(" Zone Type: Surface Straightness");
            boolstatus = annotation.GetPlanarZoneVector(ref I, ref J, ref K);
            strs.Add(" Zone Vector: " + FormatVector(I, J, K));
            break;
    }
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    //per unit length
    boolstatus = annotation.GetPerUnitLength(ref enabled, ref dist);
    strs.Add(" Per Unit Length: " + (enabled ? "True" : "False"));
    if (enabled)
    {
        strs.Add(" Per Unit Length Distance: " + FormatDouble(dist));
    }
    // the material condition modifier
    strs.Add(" Modifier: " + mcmStr((long)annotation.Modifier));
}
private void ProfileData(DimXpertTolerance annotation)
{
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
    // the datum reference frame
    DatumsStr(annotation);
}
private void SurfaceFinishData(DimXpertTolerance annotation)
{
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
}

```

```

private void TangencyData(DimXpertTangencyTolerance annotation)
{
    strs.Add("Origin Feature: " + annotation.OriginFeature.Name);
    strs.Add("");
    strs.Add("Tolerance Compartment");
    // the tolerance value
    DimXpertTolerance tol;
    tol = (DimXpertTolerance)annotation;
    strs.Add(" Tolerance: " + FormatDouble(tol.Tolerance));
}

private void DimensionToleranceData(DimXpertDimensionTolerance annotation, SolidWorks.Interop.swdimxpert.DimXpertPart
dimXpertPart)
{
    bool isAngleType = false;
    swDimXpertAnnotationType_e annoType = default(swDimXpertAnnotationType_e);
    swDimXpertPatternTreatmentType_e patternTreatment = default(swDimXpertPatternTreatmentType_e);
    double upper = 0;
    double lower = 0;
    double plus = 0;
    double minus = 0;
    bool boolstatus = false;
    DimXpertAnnotation anno;
    anno = (DimXpertAnnotation)annotation;
    annoType = anno.Type;
    isAngleType = false;
    if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween)
    {
        DistanceBetweenData((DimXpertDistanceBetweenDimTol)annotation);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween)
    {
        CompositeDistanceBetweenData((DimXpertCompositeDistanceBetweenDimTol)annotation, (DimXpertPart) dimXpertPart);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore)
    {
        DimXpertCounterBoreDimTol dimtol;
        dimtol = (DimXpertCounterBoreDimTol)annotation;
        if (dimtol.ReferenceFeature == null)
        {
            strs.Add("Reference Feature: NULL");
        }
        else
        {
            strs.Add("Reference Feature: " + dimtol.ReferenceFeature.Name);
        }
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_Depth)
    {
        DimXpertDepthDimTol dimtol;
        dimtol = (DimXpertDepthDimTol)annotation;
        if (dimtol.ReferenceFeature == null)
        {
            strs.Add("Reference Feature: NULL");
        }
        else
        {
            strs.Add("Reference Feature: " + dimtol.ReferenceFeature.Name);
        }
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter)
    {
        DimXpertCounterSinkDiameterDimTol dimtol;
    }
}

```

```

        dimtol = (DimXpertCounterSinkDiameterDimTol)annotation;
        if (dimtol.ReferenceFeature == null)
        {
            strs.Add("Reference Feature: NULL");
        }
        else
        {
            strs.Add("Reference Feature: " + dimtol.ReferenceFeature.Name);
        }
    }
else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension)
{
    DimXpertChamferDimTol chamferDimTol = default(DimXpertChamferDimTol);
    chamferDimTol = (DimXpertChamferDimTol)annotation;
    switch (chamferDimTol.ChamferType)
    {
        case swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_Angle:
            strs.Add("Chamfer Dimension Type: Angle");
            isAngleType = true;
            break;
        case swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance1:
            strs.Add("Chamfer Dimension Type: Distance 1");
            break;
        case swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance2:
            strs.Add("Chamfer Dimension Type: Distance 2");
            break;
    }
}
else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween)
{
    isAngleType = true;
    DimXpertAngleBetweenDimTol dimtol;
    dimtol = (DimXpertAngleBetweenDimTol)annotation;
    // the origin and tolerance feature
    strs.Add("Origin Feature: " + dimtol.OriginFeature.Name);
    // is supplement angle
    strs.Add("Supplement Angle: " + (dimtol.Supplement ? "True" : "False"));
}
else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle)
{
    isAngleType = true;
    DimXpertCounterSinkAngleDimTol dimtol;
    dimtol = (DimXpertCounterSinkAngleDimTol)annotation;
    if (dimtol.ReferenceFeature == null)
    {
        strs.Add("Reference Feature: NULL");
    }
    else
    {
        strs.Add("Reference Feature: " + dimtol.ReferenceFeature.Name);
    }
}
else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle)
{
    isAngleType = true;
}
else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_Radius)
{
    IDimXpertRadiusDimTol radiusDim;
    radiusDim = (IDimXpertRadiusDimTol)annotation;
    patternTreatment = radiusDim.PatternTreatment;
    //If radiusDim.patternTreatment Is Nothing Then

```

```

        //strs.Add ("Pattern Treatment: NULL")
    if (patternTreatment == swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_unknown)
        {strs.Add ("Pattern Treatment: unknown");}
    else if (patternTreatment == swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_CircularPattern)
        {strs.Add ("Pattern Treatment: Circular pattern");}
    else if (patternTreatment == swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_IndividualFeatures)
        {strs.Add ("Pattern Treatment: Individual features");}
    }
else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter)
{
    IDimXpertDiameterDimTol diaDim;
    diaDim = (IDimXpertDiameterDimTol)annotation;
    patternTreatment = diaDim.PatternTreatment;
    //If radiusDim.patternTreatment Is Nothing Then
    //strs.Add ("Pattern Treatment: NULL")
    if (patternTreatment == swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_unknown)
        { strs.Add("Pattern Treatment: unknown"); }
    else if (patternTreatment == swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_CircularPattern)
        { strs.Add("Pattern Treatment: Circular pattern"); }
    else if (patternTreatment == swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_IndividualFeatures)
        { strs.Add("Pattern Treatment: Individual features"); }

    Boolean indReq;
    Boolean envReq;
    indReq = diaDim.IndependencyRequirement;
    envReq = diaDim.EnvelopeRequirement;
    strs.Add("Independency is required: " + indReq.ToString());
    strs.Add("Envelope is required: " + envReq.ToString());
}

// conversion for radians to degrees when dimension type is angle
double dbl = 0;
if (isAngleType)
{
    dbl = 57.2957795130823;
}
else
{
    dbl = 1.0;
}

// the nominal, and upper and lower limits of size of the dimension
strs.Add("");
strs.Add("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue() * dbl));
boolstatus = annotation.GetUpperAndLowerLimit(ref upper, ref lower);
strs.Add("Get Upper Limit: " + FormatDouble(upper * dbl));
strs.Add("Get Lower Limit: " + FormatDouble(lower * dbl));
// the upper and lower tolerance value by type
switch (annotation.DimensionType)
{
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockTolerance:
        strs.Add("Dimension Type: Block Tolerance");
        break;
    // block tolerance
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal:
        DimXpertBlockTolerances blockTols = default(DimXpertBlockTolerances);
        blockTols = dimXpertPart.GetBlockTolerances();
        switch (blockTols.Type)
        {
            case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch:
                strs.Add("Dimension Type: Block Tolerance No Nominal");
                if (isAngleType)
                {
                    strs.Add("Angular Block Tolerance");
                }
                else
                {

```

```

        strs.Add("Block Tolerance Decimal Places: " +
System.String.Format("{0:N}", annotation.BlockToleranceDecimalPlaces.ToString()));
    }
    break;
    case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768:
        strs.Add("Dimension Type: General Tolerance");
        break;
    }
    break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFits:
        strs.Add("Dimension Type: Limits and Fits");
        break;
    // limits and fits tolerance
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFitsNoNominal:
        strs.Add("Dimension Type: Limits and Fits No Nominal");
        strs.Add("Limits and Fits: " + annotation.LimitsAndFitsCode);
        break;
    // limit dimension
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_LimitDimension:
        strs.Add("Dimension Type: Limit Dimension");
        boolstatus = annotation.GetUpperAndLowerLimit(ref upper, ref lower);
        strs.Add("Get Upper Limit: " + FormatDouble(upper * dbl));
        strs.Add("Get Lower Limit: " + FormatDouble(lower * dbl));
        break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MAXTolerance:
        strs.Add("Dimension Type: MAXTolerance");
        break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MINTolerance:
        strs.Add("Dimension Type: MINTolerance");
        break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_NoTolerance:
        strs.Add("Dimension Type: NoTolerance");
        break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusDimension:
        strs.Add("Dimension Type: Plus Minus Dimension");
        break;
    // plus and minus dimension
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal:
        strs.Add("Dimension Type: Plus Minus No Nominal");
        boolstatus = annotation.GetPlusAndMinusTolerance(ref plus, ref minus);
        strs.Add("Plus Tolerance: " + FormatDouble(plus * dbl));
        strs.Add("Minus Tolerance: " + FormatDouble(minus * dbl));
        break;
    }
}
private void DistanceBetweenData(DimXpertDistanceBetweenDimTol annotation)
{
    DimXpertFeature feature = default(DimXpertFeature);
    swDimXpertDistanceFosUsage_e featureFosUsage = default(swDimXpertDistanceFosUsage_e);
    double I = 0;
    double J = 0;
    double K = 0;
    bool boolstatus = false;
    feature = null;
    // the origin and tolerance feature along with their feature of size usage (min, max, center)
    boolstatus = annotation.GetOriginFeature(ref feature, ref featureFosUsage);
    strs.Add("");
    strs.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage));
    boolstatus = annotation.GetFeature(ref feature, ref featureFosUsage);
    strs.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage));
    // The direction vector
    boolstatus = annotation.GetDirectionVector(ref I, ref J, ref K);

```

```

        strs.Add("");
        strs.Add("Direction Vector: " + FormatVector(I, J, K));
    }
    private void CompositeDistanceBetweenData(DimXpertCompositeDistanceBetweenDimTol annotation,
SolidWorks.Interop.swdimxpert.DimXpertPart dimXpertPart)
    {
        DimXpertFeature feature = default(DimXpertFeature);
        swDimXpertDistanceFosUsage_e featureFosUsage = default(swDimXpertDistanceFosUsage_e);
        double plus = 0;
        double minus = 0;
        double I = 0;
        double J = 0;
        double K = 0;
        bool boolstatus = false;
        bool isAngleType = false;
        DimXpertBlockTolerances blockTols = default(DimXpertBlockTolerances);
        isAngleType = false;
        // conversion for radians to degrees when dimension type is angle
        double dbl = 0;
        if (isAngleType)
        {
            dbl = 57.2957795130823;
        }
        else
        {
            dbl = 1.0;
        }
        feature = null;
        // the origin and tolerance feature along with their feature of size usage (min, max, center)
        boolstatus = annotation.GetOriginFeature(ref feature, ref featureFosUsage);
        strs.Add("");
        strs.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage));
        boolstatus = annotation.GetFeature(ref feature, ref featureFosUsage);
        strs.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage));
        // the pattern locating feature
        boolstatus = annotation.GetIntraFeature(ref feature, ref featureFosUsage);
        strs.Add("Pattern Locating Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage));
        // The direction vector
        boolstatus = annotation.GetDirectionVector(ref I, ref J, ref K);
        strs.Add("");
        strs.Add("Direction Vector: " + FormatVector(I, J, K));

        // the upper and lower tolerance value for the pattern location by type
        DimXpertDimensionTolerance dimTol = default(DimXpertDimensionTolerance);
        dimTol = null;
        dimTol = (DimXpertDimensionTolerance)annotation;
        switch (dimTol.DimensionType)
        {
            // plus and minus dimension
            case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal:
                strs.Add("Pattern Locating Dimension Type: Plus Minus No Nominal");
                boolstatus = dimTol.GetPlusAndMinusTolerance(ref plus, ref minus);
                strs.Add(" Plus Tolerance: " + FormatDouble(plus));
                strs.Add(" Minus Tolerance: " + FormatDouble(minus));
                break;
            // block tolerance
            case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal:
                blockTols = dimXpertPart.GetBlockTolerances();
                switch (blockTols.Type)
                {
                    case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch:
                        strs.Add("Pattern Locating Dimension Type: Block Tolerance No Nominal");

```

```

        strs.Add(" Block Tolerance Decimal Places: " +
System.String.Format("{0:N}", dimTol.BlockToleranceDecimalPlaces.ToString()));
        break;
        case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768:
            strs.Add("Pattern Locating Dimension Type: General Tolerance");
            break;
    }
    break;
}
// the upper and lower tolerance value for the feature to feature location by type
switch (annotation.DimensionTypeIntraFeature)
{
    // plus and minus dimension
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal:
        strs.Add("Feature Locating Dimension Type: Plus Minus No Nominal");
        boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(ref plus, ref minus);
        strs.Add(" Plus Tolerance: " + FormatDouble(plus * dbl));
        strs.Add(" Minus Tolerance: " + FormatDouble(minus * dbl));
        break;
    // block tolerance
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal:
        blockTols = dimXpertPart.GetBlockTolerances();
        switch (blockTols.Type)
        {
            case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch:
                strs.Add("Feature locating Dimension Type: Block Tolerance No Nominal");
                strs.Add(" Block Tolerance Decimal Places: " +
System.String.Format("{0:N}", annotation.BlockToleranceDecimalPlacesIntraFeature.ToString()));
                break;
            case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768:
                strs.Add("Feature locating Dimension Type: General Tolerance");
                break;
        }
        break;
    }
}
private void DatumsStr(DimXpertTolerance annotation)
{
    strs.Add("");
    strs.Add("Datums:");
    datumStr((object[])annotation.GetPrimaryDatums(), (int[])annotation.GetPrimaryDatumModifiers(), " Primary:");
    datumStr((object[])annotation.GetSecondaryDatums(), (int[])annotation.GetSecondaryDatumModifiers(), " Secondary:");
    datumStr((object[])annotation.GetTertiaryDatums(), (int[])annotation.GetTertiaryDatumModifiers(), " Tertiary:");
}
private void datumStr(object[] dats, int[] mods, string datumOrder)
{
    long I = 0;
    string str = null;
    long mcm = 0;

    if ((dats == null))
    {
        return;
    }
    str = "";
    for (I = dats.GetLowerBound(0); I <= dats.GetUpperBound(0); I++)
    {
        mcm = (long)mods[I];
        DimXpertDatum datum;
        datum = (DimXpertDatum)dats[I];
        str = str + " " + datum.Identifier + " @ " + mcmStr(mcm);
    }
}

```

```

    }
    if (System.String.Compare(str, "") > 0)
    {
        strs.Add(datumOrder + str);
    }
    else
    {
        strs.Add(datumOrder + " <none>");
    }
}
// returns a string containing the height of the projected tolerance zone
private void FormatProjectedZone(bool enabled, double height)
{
    if (enabled == true)
    {
        strs.Add(" Projected Zone: True");
        strs.Add(" Zone Height: " + FormatDouble(height));
    }
    else
    {
        strs.Add(" Projected Zone: False");
    }
}
private string mcmStr(long mcm)
{
    string str = null;
    str = "";
    switch ((swDimXpertMaterialConditionModifier_e)mcm)
    {
        case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_LMC:
            str = "LMC";
            break;
        case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_MMC:
            str = "MMC";
            break;
        case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_NoMCM:
            str = "NoMCM";
            break;
        case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_RFS:
            str = "RFS";
            break;
    }
    return str;
}
// returns a string containing the type of position tolerance zone used
private string PositionZoneType(long typ)
{
    string str = null;
    str = "";
    switch ((swDimXpertPositionZoneType_e)typ)
    {
        case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_CylindricalPosition:
            str = "Cylindrical";
            break;
        case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition:
            str = "Planar";
            break;
        case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_SphericalPosition:
            str = "Spherical";
            break;
        case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_Boundary:
            str = "Boundary";

```

```

        break;
    case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionArc:
        str = "RadialPositionArc";
        break;
    case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionPlanar:
        str = "RadialPositionPlanar";
        break;
    default:
        str = "N/A";
        break;
    }
    return str;
}

// returns a string containing the names of the SW display entities
private string DisplayEntity(DimXpertAnnotation annotation)
{
    string str = null;
    object dispEnt = null;
    Annotation swAnnot = default(Annotation);
    str = "";
    dispEnt = annotation.GetDisplayEntity();
    if ((dispEnt != null))
    {
        if (dispEnt is Annotation)
        {
            swAnnot = (Annotation)dispEnt;
            str = (String)swAnnot.GetName();
        }
    }
    return str;
}

// returns a string containing the feature of size usage (min, max, center)
private string FosUsage(swDimXpertDistanceFosUsage_e value)
{
    string str = null;
    str = "";
    switch (value)
    {
        case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_Center:
            str = "Center";
            break;
        case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MaximumSide:
            str = "Max";
            break;
        case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MinimumSide:
            str = "Min";
            break;
        default:
            str = "N/A";
            break;
    }
    return str;
}

private string FormatVector(double I, double J, double K)
{
    string str = null;
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K);
    return str;
}

private string FormatDouble(double value)
{
    string str = null;

```

```

        str = System.String.Format("{0:D}", value.ToString());
        return str;
    }
    private string RadiansToDegrees(double value)
    {
        string str = null;
        str = System.String.Format("{0:D}", (value * 57.2957795130823).ToString());
        return str;
    }
    private string annotationTypeNameFromObject(DimXpertAnnotation anno)
    {
        return annotationTypeNameFromTypeNumber(anno.Type);
    }
    private string annotationTypeNameFromTypeNumber(swDimXpertAnnotationType_e annoTypeIndex)
    {
        string functionReturnValue = null;
        switch (annoTypeIndex)
        {
            case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween:
                functionReturnValue = "DistanceBetween Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore:
                functionReturnValue = "CounterBore Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_Depth:
                functionReturnValue = "Depth Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter:
                functionReturnValue = "CounterSinkDiameter Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension:
                functionReturnValue = "ChamferDimension Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween:
                functionReturnValue = "AngleBetween Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle:
                functionReturnValue = "CounterSinkAngle Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle:
                functionReturnValue = "ConeAngle Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter:
                functionReturnValue = "Diameter Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_Length:
                functionReturnValue = "Length Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_Radius:
                functionReturnValue = "Radius Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_Width:
                functionReturnValue = "Width Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween:
                functionReturnValue = "CompositeDistanceBetween Dim";
                break;
            case swDimXpertAnnotationType_e.swDimXpertDatum:
                functionReturnValue = "Datum";
                break;
            case swDimXpertAnnotationType_e.swDimXpertGeoTol_Position:
                functionReturnValue = "Position Tol";

```

```

        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition:
        functionReturnValue = "CompositePosition Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry:
        functionReturnValue = "Symmetry Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity:
        functionReturnValue = "Concentricity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile:
        functionReturnValue = "LineProfile Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile:
        functionReturnValue = "CompositeLineProfile Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile:
        functionReturnValue = "SurfaceProfile Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile:
        functionReturnValue = "CompositeSurfaceProfile Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity:
        functionReturnValue = "Angularity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism:
        functionReturnValue = "Parallelism Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity:
        functionReturnValue = "Perpendicularity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout:
        functionReturnValue = "TotalRunout Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout:
        functionReturnValue = "CircularRunout Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness:
        functionReturnValue = "Flatness Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity:
        functionReturnValue = "Circularity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity:
        functionReturnValue = "Cylindricity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness:
        functionReturnValue = "Straightness Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency:
        functionReturnValue = "Tangency Tol";
        break;
    default:
        functionReturnValue = "<unknown> " + annoTypeIndex;
        break;
}
return functionReturnValue;
}
}

```

DimXpert Feature Data Module Example (C#)

```
// This module is a component of
// Get DimXpert Features and Annotations in a Model Example \(C#\).

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swdimxpert;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Collections.ObjectModel;
public class DimXpertFeatureData
{
    // Returns a collection of Strings containing the features data
    private Collection<string> strs = new Collection<string>();
    public void Clear(Collection<string> strs)
    {
        strs.Remove(strs[strs.Count]);
        if (!(strs.Count == 0))
        {
            Clear(strs);
        }
    }
    public Collection<string> FeatureData(DimXpertFeature feature)
    {
        SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e featureType =
default(SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e);
        if (!(strs.Count == 0))
        {
            Clear(strs);
        }
        // General info, including the feature name, face id, and suppressed status
        FeatInfo(feature);

        featureType = feature.Type;
        // DimXpert plane
        if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Plane)
        {
            PlaneData((DimXpertPlaneFeature)feature);
        }
        // DimXpert cylinder or boss. Also used for the sub-feature of a
        // compound hole (simple hole, counterbore, countersink...)
        else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Cylinder)
        {
            CylinderData((DimXpertCylinderFeature)feature);
        }
        // DimXpert cone
        else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Cone)
        {
            ConeData((DimXpertConeFeature)feature);
        }
        // DimXpert pocket or extrude
        else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Extrude)
        {
            ExtrudeData((DimXpertExtrudeFeature)feature);
        }
        // DimXpert fillet
        else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Fillet)
        {

```

```

{
    FilletData((DimXpertFilletFeature)feature);
}
// DimXpert chamfer
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Chamfer)
{
    ChamferData((DimXpertChamferFeature)feature);
}
// DimXpert hole type (Simple, counterbore, countersink...)
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole)
{
    CompoundHoleData((DimXpertCompoundHoleFeature)feature);
}
// DimXpert width
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth)
{
    CompoundWidthData((DimXpertCompoundWidthFeature)feature);
}
// DimXpert notch
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch)
{
    CompoundNotchData((DimXpertCompoundNotchFeature)feature);
}
// DimXpert slot
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D)
{
    CompoundSlot3dData((DimXpertCompoundClosedSlot3DFeature)feature);
}
// DimXpert intersect point
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint)
{
    IntersectPointData((DimXpertIntersectPointFeature)feature);
}
// DimXpert intersect line
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine)
{
    IntersectLineData((DimXpertIntersectLineFeature)feature);
}
// DimXpert intersect circle
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle)
{
    IntersectCircleData((DimXpertIntersectCircleFeature)feature);
}
// DimXpert intersect plane
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane)
{
    IntersectPlaneData((DimXpertIntersectPlaneFeature)feature);
}
// DimXpert pattern (hole, slot, notch...)
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Pattern)
{
    PatternData((DimXpertPatternFeature)feature);
}
// DimXpert sphere
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Sphere)
{
    SphereData((DimXpertSphereFeature)feature);
}
// DimXpert best fit plane
else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane)
{
    BestfitPlaneData((DimXpertBestfitPlaneFeature)feature);
}

```

```

    }
    // DimXpert surface
    else if (featureType == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Surface)
    {
        SurfaceData((DimXpertFeature)feature);
    }
    return str;
}

private void FeatInfo(DimXpertFeature feature)
{
    string featureType = null;
    Feature modelObj = default(Feature);
    str.Add("Name: " + feature.Name);
    featureType = featureTypeNameFromObject(feature);
    str.Add("Type: " + featureType);
    str.Add(GetFaceId(feature));
    str.Add("IsSuppressed: " + (feature.IsSuppressed() ? "True" : "False"));
    modelObj = (Feature)feature.GetModelFeature();
    if ((modelObj != null))
    {
        str.Add("ModelFeature: " + modelObj.Name + " (" + modelObj.GetTypeName2() + ")");
    }
}

private void PlaneData(DimXpertPlaneFeature feature)
{
    bool boolstatus = false;
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    // list the plane's nominal point and vector
    str.Add("");
    str.Add("Nominal Plane:");
    boolstatus = feature.GetNominalPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
    str.Add(" Point: " + FormatABC(X, Y, Z));
    str.Add(" Vector: " + FormatABC(I, J, K));
}

private void CylinderData(DimXpertCylinderFeature feature)
{
    bool boolstatus = false;
    bool isThreaded = false;
    string threadDesignation = null;
    double threadDepth = 0;
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    double R = 0;
    // isInner status, is the feature a hole or boss?
    str.Add("");
    str.Add("IsInner: " + (feature.Inner ? "True" : "False"));
    threadDesignation = "";
    // isThreaded status
    boolstatus = feature.GetThread(ref isThreaded, ref threadDesignation, ref threadDepth);
    str.Add("IsThreaded: " + (isThreaded ? "True" : "False"));
    // thread designation and depth
    if (isThreaded)
    {

```

```

        strs.Add("Thread Designation: " + threadDesignation);
        strs.Add("Thread Depth: " + FormatDouble(threadDepth));
    }
    // the cylinder's diameter, axis point, and vector
    strs.Add("");
    strs.Add("Nominal Cylinder:");
    boolstatus = feature.GetNominalCylinder(ref R, ref X, ref Y, ref Z, ref I, ref J, ref K);
    strs.Add(" Diameter: " + FormatDouble(R * 2));
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Vector: " + FormatABC(I, J, K));
    // the nominal top plane of the cylinder, which is the highest point
    // of the cylinder's boundary projected onto its axis
    strs.Add("");
    strs.Add("Nominal Top Plane:");
    boolstatus = feature.GetNominalTopPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Vector: " + FormatABC(I, J, K));
    // the nominal bottom plane of the cylinder, which is the lowest point
    // of the cylinder's boundary projected onto its axis
    strs.Add("");
    strs.Add("Nominal Bottom Plane:");
    boolstatus = feature.GetNominalBottomPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Vector: " + FormatABC(I, J, K));
}
private void ConeData(DimXpertConeFeature feature)
{
    bool boolstatus = false;
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    double Angle = 0;
    //isInner status
    strs.Add("");
    strs.Add("IsInner: " + (feature.Inner ? "True" : "False"));
    //the nominal cone's angle, axis point, and vector
    strs.Add("");
    strs.Add("Nominal Cone:");
    boolstatus = feature.GetNominalCone(ref Angle, ref X, ref Y, ref Z, ref I, ref J, ref K);
    strs.Add(" Cone Angle: " + RadiansToDegrees(Angle));
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Vector: " + FormatABC(I, J, K));
    // the nominal top plane of the cone, which is the highest point
    // of the cone's boundary projected onto its axis
    strs.Add("");
    strs.Add("Nominal Top Plane:");
    boolstatus = feature.GetNominalTopPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Vector: " + FormatABC(I, J, K));
    // the nominal bottom plane of the cone, which is the lowest point
    // of the cone's boundary projected onto its axis
    strs.Add("");
    strs.Add("Nominal Bottom Plane:");
    boolstatus = feature.GetNominalBottomPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Vector: " + FormatABC(I, J, K));
}
private void FilletData(DimXpertFilletFeature feature)
{

```

```

//isInner status
strs.Add("");
strs.Add("IsInner: " + (feature.Inner ? "True" : "False"));
//the fillet radius
strs.Add("Radius: " + FormatDouble(feature.Radius));
}
private void ChamferData(DimXpertChamferFeature feature)
{
    strs.Add("");
    // chamfer dimension scheme, which can be a distance and an angle, or 2 distances
    switch (feature.ChamferType)
    {
        case SolidWorks.Interop.swdimxpert.swDimXpertChamferType_e.swDimXpertChamferType_DistanceAngle:
            strs.Add("Chamfer Dimension: Distance and Angle");
            strs.Add(" Distance: " + FormatDouble(feature.Distance1));
            strs.Add(" Angle: " + RadiansToDegrees(feature.Angle));
            break;
        case SolidWorks.Interop.swdimxpert.swDimXpertChamferType_e.swDimXpertChamferType_DistanceDistance:
            strs.Add("Chamfer Dimension: 2 Distances");
            strs.Add(" Distance1: " + FormatDouble(feature.Distance1));
            strs.Add(" Distance2: " + FormatDouble(feature.Distance2));
            break;
        case SolidWorks.Interop.swdimxpert.swDimXpertChamferType_e.swDimXpertChamferType_Vertex:
            strs.Add("Chamfer Type: Vertex");
            break;
    }
}
private void CompoundHoleData(DimXpertCompoundHoleFeature feature)
{
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    double R = 0;
    long index = 0;
    DimXpertFeature featObj = default(DimXpertFeature);
    bool boolstatus = false;
    strs.Add("");
    // the type of compound hole
    string typ = null;
    typ = "";
    switch (feature.CompoundHoleType)
    {
        case SolidWorks.Interop.swdimxpert.swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Counterbore:
            typ = "Counterbore";
            break;
        case SolidWorks.Interop.swdimxpert.swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Countersink:
            typ = "Countersink";
            break;
        case SolidWorks.Interop.swdimxpert.swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Compound:
            typ = "Compound";
            break;
        case SolidWorks.Interop.swdimxpert.swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Simple:
            typ = "Simple";
            break;
    }
    strs.Add("Hole Type: " + typ);
    // is blind status
    strs.Add("IsBlind: " + (feature.Blind ? "True" : "False"));
    // reference plane, which is the basis for all depth tolerances

```

```

if ((feature.GetReferenceFeature() != null))
{
    strs.Add("Reference Feature: " + feature.GetReferenceFeature().Name);
}
else
{
    strs.Add("Reference Feature: isNothing");
}
// bottom feature, used for the depth of blind holes
if ((feature.GetBottomFeature() != null))
{
    strs.Add("Bottom Feature: " + feature.GetBottomFeature().Name);
}
else
{
    strs.Add("Bottom Feature: isNothing");
}
//get the collection of sub-features
object[] subFeats = null;
subFeats = (object[])feature.GetSubFeatures();
// the collection of sub-features that make up the hole
strs.Add("" + "Sub-Features:");
if (!(subFeats == null))
{
    for (index = subFeats.GetLowerBound(0); index <= subFeats.GetUpperBound(0); index++)
    {
        featObj = (DimXpertFeature)subFeats[index];
        strs.Add(" " + featObj.Name + "");
    }
    // the location of the hole based on the first cylinder in the list of sub-features
    for (index = subFeats.GetLowerBound(0); index <= subFeats.GetUpperBound(0); index++)
    {
        featObj = (DimXpertFeature)subFeats[index];
        if (featObj.Type == SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Cylinder)
        {
            strs.Add("");
            DimXpertCylinderFeature cylFeat = default(DimXpertCylinderFeature);
            cylFeat = (DimXpertCylinderFeature)featObj;
            boolstatus = cylFeat.GetNominalCylinder(ref R, ref X, ref Y, ref Z, ref I, ref J, ref K);
            // the diameter of the hole for type simple
            switch (feature.CompoundHoleType)
            {
                case SolidWorks.Interop.swdimxpert.swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Simple:
                    strs.Add("Diameter: " + FormatDouble(R * 2));
                    break;
            }
            strs.Add("Nominal Axis:");
            strs.Add(" Point: " + FormatABC(X, Y, Z));
            strs.Add(" Vector: " + FormatABC(I, J, K));
            index = subFeats.GetUpperBound(0);
        }
    }
}
}
private void CompoundWidthData(DimXpertCompoundWidthFeature feature)
{
    DimXpertPlaneFeature plane1 = default(DimXpertPlaneFeature);
    DimXpertPlaneFeature plane2 = default(DimXpertPlaneFeature);
    bool boolstatus = false;
    double X = 0;
    double Y = 0;
    double Z = 0;

```

```

double I = 0;
double J = 0;
double K = 0;
double width = 0;
// the two side features
strs.Add("");
strs.Add("Features:");
plane1 = null;
plane2 = null;
boolstatus = feature.GetPlaneFeatures(ref plane1, ref plane2);
if ((plane1 != null) & (plane2 != null))
{
    DimXpertFeature feat1 = (DimXpertFeature)plane1;
    DimXpertFeature feat2 = (DimXpertFeature)plane2;
    strs.Add(" Plane1: " + feat1.Name);
    strs.Add(" Plane2: " + feat2.Name);
}
// isInner status
strs.Add("IsInner: " + (feature.Inner ? "True" : "False"));
// the nominal width and mid-plane
strs.Add("");
strs.Add("Nominal Width:");
boolstatus = feature.GetNominalCompoundWidth(ref width, ref X, ref Y, ref Z, ref I, ref J, ref K);
strs.Add(" Width: " + FormatDouble(width));
strs.Add(" Point: " + FormatABC(X, Y, Z));
strs.Add(" Vector: " + FormatABC(I, J, K));
// the width's longitudinal direction
boolstatus = feature.GetNominalLongitude(ref I, ref J, ref K);
strs.Add("Longitudinal Direction: " + FormatABC(I, J, K));
}
private void CompoundSlot3dData(DimXpertCompoundClosedSlot3DFeature feature)
{
    DimXpertPlaneFeature plane1 = default(DimXpertPlaneFeature);
    DimXpertPlaneFeature plane2 = default(DimXpertPlaneFeature);
    DimXpertFeature end1 = default(DimXpertFeature);
    DimXpertFeature end2 = default(DimXpertFeature);
    bool boolstatus = false;
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    double longitudeI = 0;
    double longitudeJ = 0;
    double longitudeK = 0;
    double width = 0;
    double length = 0;
    //isInner status
    strs.Add("");

    //isBlind status
    strs.Add("IsBlind: " + (feature.Blind ? "True" : "False"));
    // the two side features
    strs.Add("Features:");
    plane1 = null;
    plane2 = null;
    boolstatus = feature.GetPlaneFeatures(ref plane1, ref plane2);
    DimXpertFeature feat1 = (DimXpertFeature)plane1;
    DimXpertFeature feat2 = (DimXpertFeature)plane2;
    strs.Add(" Side1: " + feat1.Name);
    strs.Add(" Side2: " + feat2.Name);
}

```

```

//the two end features, which can be cylinders or planes
end1 = null;
end2 = null;
boolstatus = feature.GetEndFeatures(ref end1, ref end2);
strs.Add(" End1: " + end1.Name);
strs.Add(" End2: " + end2.Name);
// bottom feature, used for the depth of blind holes
if ((feature.GetBottomFeature() != null))
{
    strs.Add("Bottom Feature: " + feature.GetBottomFeature().Name);
}
else
{
    strs.Add("Bottom Feature: isNothing");
}
// the size and location of the nominal slot
boolstatus = feature.GetNominalClosedSlot(ref width, ref length, ref X, ref Y, ref Z, ref I, ref J, ref K, ref longitudeI,
ref longitudeJ,
ref longitudeK);
strs.Add("");
strs.Add("Nominal Slot:");
strs.Add(" Width: " + FormatDouble(width));
strs.Add(" Length: " + FormatDouble(length));
strs.Add(" Axis Point: " + FormatABC(X, Y, Z));
strs.Add(" Axis Vector: " + FormatABC(I, J, K));
strs.Add(" Longitudinal Vector: " + FormatABC(longitudeI, longitudeJ, longitudeK));
// the nominal top plane of the slot
strs.Add("");
strs.Add("Nominal Top Plane:");
boolstatus = feature.GetNominalTopPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
strs.Add(" Point: " + FormatABC(X, Y, Z));
strs.Add(" Vector: " + FormatABC(I, J, K));
// the nominal bottom plane of the slot
strs.Add("");
strs.Add("Nominal Bottom Plane:");
boolstatus = feature.GetNominalBottomPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
strs.Add(" Point: " + FormatABC(X, Y, Z));
strs.Add(" Vector: " + FormatABC(I, J, K));
}
private void CompoundNotchData(DimXpertCompoundNotchFeature feature)
{
    DimXpertPlaneFeature plane1 = default(DimXpertPlaneFeature);
    DimXpertPlaneFeature plane2 = default(DimXpertPlaneFeature);
    bool boolstatus = false;
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    double longitudeI = 0;
    double longitudeJ = 0;
    double longitudeK = 0;
    double width = 0;
    double length = 0;
    // isInner status
    strs.Add("");

    // isBlind status
    strs.Add("IsBlind: " + (feature.Blind ? "True" : "False"));
    // the reference feature or top of the notch
    strs.Add("Top Reference Feature: " + feature.GetTopReferenceFeature().Name);
    // the open side reference feature, which bounds the notch

```

```

    strs.Add("Open Side Reference Feature: " + feature.GetOpenSideReferenceFeature().Name);
    // the two side features
    plane1 = null;
    plane2 = null;
    boolstatus = feature.GetPlaneFeatures(ref plane1, ref plane2);
    strs.Add("Features: " + "");
    DimXpertFeature feat1 = (DimXpertFeature)plane1;
    DimXpertFeature feat2 = (DimXpertFeature)plane2;
    strs.Add(" Side1: " + feat1.Name);
    strs.Add(" Side2: " + feat2.Name);
    // the end features, which can be a cylinder or a plane
    if ((feature.GetEndFeature() != null))
    {
        strs.Add(" End Feature: " + feature.GetEndFeature().Name);
    }
    else
    {
        strs.Add(" End Feature: isNothing");
    }
    // bottom feature, used for the depth of blind holes
    if ((feature.GetBottomFeature() != null))
    {
        strs.Add(" Bottom Feature: " + feature.GetBottomFeature().Name);
    }
    else
    {
        strs.Add(" Bottom Feature: isNothing");
    }
    // the nominal size and location of the notch
    boolstatus = feature.GetNominalNotch(ref width, ref length, ref X, ref Y, ref Z, ref I, ref J, ref K, ref longitudeI, ref
longitudeJ,
    ref longitudeK);
    strs.Add("");
    strs.Add("Nominal Notch:");
    strs.Add(" Width: " + FormatDouble(width));
    strs.Add(" Length: " + FormatDouble(length));
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Plane Vector: " + FormatABC(I, J, K));
    strs.Add(" Longitudinal Vector: " + FormatABC(longitudeI, longitudeJ, longitudeK));
    // the nominal bottom plane of the notch
    strs.Add("");
    strs.Add("Nominal Bottom Plane:");
    boolstatus = feature.GetNominalBottomPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Vector: " + FormatABC(I, J, K));
}
private void ExtrudeData(DimXpertExtrudeFeature feature)
{
    long index = 0;
    object[] wallFeats = null;
    DimXpertFeature featObj = default(DimXpertFeature);
    // isBlind status
    strs.Add("");
    strs.Add("IsBlind: " + (feature.Blind ? "True" : "False"));
    // the reference feature or top plane
    strs.Add("Reference Feature: " + feature.GetReferenceFeature().Name);
    // the bottom feature
    strs.Add("Bottom Feature: " + feature.GetBottomFeature().Name);
    // the top blends (fillets or chamfers)
    if ((feature.GetTopBlends() != null))
    {
        strs.Add("Top Blends: " + feature.GetTopBlends().Name);
    }
}

```

```

else
{
    strs.Add("Top Blends: Is_Nothing");
}
// the bottom blends
if ((feature.GetBottomBlends() != null))
{
    strs.Add("Bottom Blends: " + feature.GetBottomBlends().Name);
}
else
{
    strs.Add("Bottom Blends: Is_Nothing");
}
// the wall features of the pocket or extrude
strs.Add("Wall Features:");
wallFeats = (object[])feature.GetSubFeatures();
if ((wallFeats != null))
{
    for (index = wallFeats.GetLowerBound(0); index <= wallFeats.GetUpperBound(0); index++)
    {
        featObj = (DimXpertFeature)wallFeats[index];
        strs.Add(" " + featObj.Name);
    }
}
}
private void IntersectPointData(DimXpertIntersectPointFeature feature)
{
    DimXpertFeature feature1 = default(DimXpertFeature);
    DimXpertFeature feature2 = default(DimXpertFeature);
    bool boolstatus = false;
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    // the two features that make up the intersect point
    feature1 = null;
    feature2 = null;
    boolstatus = feature.GetFeatures(ref feature1, ref feature2);
    if ((feature1 != null) & (feature2 != null))
    {
        strs.Add("");
        strs.Add("Sub-Features: " + "");

        strs.Add(" " + feature1.Name);
        strs.Add(" " + feature2.Name);
    }
    // the nominal point
    boolstatus = feature.GetNominalPoint(ref X, ref Y, ref Z);
    strs.Add("Intersect Point: " + FormatABC(X, Y, Z));
    // the nominal vector
    boolstatus = feature.GetNominalVector(ref I, ref J, ref K);
    strs.Add("Point Vector: " + FormatABC(I, J, K));
}
private void IntersectLineData(DimXpertIntersectLineFeature feature)
{
    DimXpertPlaneFeature plane1 = default(DimXpertPlaneFeature);
    DimXpertPlaneFeature plane2 = default(DimXpertPlaneFeature);
    bool boolstatus = false;
    double X = 0;
    double Y = 0;

```

```

double Z = 0;
double I = 0;
double J = 0;
double K = 0;
// the two features that make up the intersect line
plane1 = null;
plane2 = null;
boolstatus = feature.GetPlaneFeatures(ref plane1, ref plane2);
if ((plane1 != null) & (plane2 != null))
{
    strs.Add("");
    strs.Add("Sub-Features: " + "");
    DimXpertFeature feat1 = (DimXpertFeature)plane1;
    DimXpertFeature feat2 = (DimXpertFeature)plane2;
    strs.Add(" " + feat1.Name);
    strs.Add(" " + feat2.Name);
}
// the nominal line
boolstatus = feature.GetNominalLine(ref X, ref Y, ref Z, ref I, ref J, ref K);
strs.Add("Nominal Line:");
strs.Add(" Point: " + FormatABC(X, Y, Z));
strs.Add(" Vector: " + FormatABC(I, J, K));
}
private void IntersectCircleData(DimXpertIntersectCircleFeature feature)
{
    DimXpertPlaneFeature plane1 = default(DimXpertPlaneFeature);
    bool boolstatus = false;
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    double R = 0;
    plane1 = null;
    boolstatus = feature.GetPlaneFeature(ref plane1);
    if ((plane1 != null))
    {
        DimXpertFeature feat = (DimXpertFeature)plane1;
        strs.Add("Plane Feature: " + feat.Name);
    }
    boolstatus = feature.GetNominalCircle(ref R, ref X, ref Y, ref Z, ref I, ref J, ref K);
    strs.Add("Nominal Circle:");
    strs.Add(" Diameter: " + FormatDouble(R * 2));
    strs.Add(" Point: " + FormatABC(X, Y, Z));
    strs.Add(" Vector: " + FormatABC(I, J, K));
}
private void IntersectPlaneData(DimXpertIntersectPlaneFeature feature)
{
    DimXpertFeature feature1 = default(DimXpertFeature);
    DimXpertFeature feature2 = default(DimXpertFeature);
    bool boolstatus = false;
    double X = 0;
    double Y = 0;
    double Z = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    // the two features that make up the intersect plane
    feature1 = null;
    feature2 = null;

```

```

boolstatus = feature.GetFeatures(ref feature1, ref feature2);
if ((feature1 != null) & (feature2 != null))
{
    strs.Add("");
    strs.Add("Sub-Features:");
    strs.Add(" " + feature1.Name);
    strs.Add(" " + feature2.Name);
}
// the nominal plane
strs.Add("Nominal Plane");
boolstatus = feature.GetNominalPlane(ref X, ref Y, ref Z, ref I, ref J, ref K);
strs.Add(" Point: " + FormatABC(X, Y, Z));
strs.Add(" Vector: " + FormatABC(I, J, K));
}
private void PatternData(DimXpertPatternFeature feature)
{
    object[] afeats = null;
    DimXpertFeature featObj = default(DimXpertFeature);
    long index = 0;
    strs.Add("");
    strs.Add("Pattern Type: " + feature.TypeNameFromTypeNumber(feature.PatternType));
    // list the collection of features
    afeats = (object[])feature.GetSubFeatures();
    strs.Add("Sub-Features:");
    if ((afeats != null))
    {
        for (index = afeats.GetLowerBound(0); index <= afeats.GetUpperBound(0); index++)
        {
            featObj = (DimXpertFeature)afeats[index];
            strs.Add(" " + featObj.Name);
        }
    }
}
private void SphereData(DimXpertSphereFeature feature)
{
    strs.Add("");
    bool boolstatus = false;
    double R = 0;
    double X = 0;
    double Y = 0;
    double Z = 0;
    //isInner status
    strs.Add("");
    strs.Add("IsInner: " + (feature.Inner ? "True" : "False"));
    //the nominal sphere's radius and x, y, and z
    strs.Add("");
    strs.Add("Nominal Sphere:");
    boolstatus = feature.GetNominalSphere(ref R, ref X, ref Y, ref Z);
    strs.Add(" Radius: " + R);
    strs.Add(" Point: " + FormatABC(X, Y, Z));
}
private void BestfitPlaneData(DimXpertBestfitPlaneFeature feature)
{
    strs.Add("");
    object[] afeats = null;
    DimXpertFeature featObj = default(DimXpertFeature);
    long index = 0;
    // list the collection of features
    afeats = (object[])feature.GetSubFeatures();
    strs.Add("Sub-Features:");
    if ((afeats != null))
    {

```

```

        for (index = afeats.GetLowerBound(0); index <= afeats.GetUpperBound(0); index++)
        {
            featObj = (DimXpertFeature)afeats[index];
            strs.Add(" " + featObj.Name);
        }
    }
}
private void SurfaceData(DimXpertFeature feature)
{
    strs.Add("");
    strs.Add("Surface Feature...");
}
private string FormatABC(double a, double b, double c)
{
    string str = null;
    str = FormatDouble(a) + ", " + FormatDouble(b) + ", " + FormatDouble(c);
    return str;
}
private string FormatDouble(double value)
{
    string str = null;
    str = System.String.Format("{0:D}", value.ToString());
    return str;
}
private string RadiansToDegrees(double value)
{
    string str = null;
    str = System.String.Format("{0:D}", (value * 57.2957795130823).ToString());
    return str;
}
private string GetFaceId(DimXpertFeature feature)
{
    string str = null;
    object[] vFaceArr = null;
    Face2 swFace2 = null;
    long lFeatId = 0;
    long I = 0;
    str = "Face ID:";
    vFaceArr = (Object[])feature.GetFaces();
    if ((vFaceArr != null))
    {
        for (I = vFaceArr.GetUpperBound(0); I <= vFaceArr.GetUpperBound(0); I++)
        {
            swFace2 = (Face2)vFaceArr[I];
            lFeatId = swFace2.GetFeatureId();
            str = str + " " + System.String.Format("{0:N}", lFeatId.ToString());
        }
    }
    else
    {
        str = str + " <Nothing>";
    }
    return str;
}
private string featureTypeNameFromObject(DimXpertFeature feature)
{
    return featureTypeNameFromTypeNumber(feature.Type);
}
private string featureTypeNameFromTypeNumber(SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e featureTypeIndex)
{
    string functionReturnValue = null;
    switch (featureTypeIndex)

```

```

{
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Plane:
        functionReturnValue = "Plane";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Cylinder:
        functionReturnValue = "Cylinder";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Cone:
        functionReturnValue = "Cone";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Extrude:
        functionReturnValue = "Extrude";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Fillet:
        functionReturnValue = "Fillet";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Chamfer:
        functionReturnValue = "Chamfer";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole:
        functionReturnValue = "Compound Hole";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth:
        functionReturnValue = "Compound Width";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch:
        functionReturnValue = "Compound Notch";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D:
        functionReturnValue = "Compound Closed Slot";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint:
        functionReturnValue = "IntersectPoint";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine:
        functionReturnValue = "IntersectLine";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle:
        functionReturnValue = "IntersectCircle";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane:
        functionReturnValue = "IntersectPlane";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Pattern:
        functionReturnValue = "Pattern";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Sphere:
        functionReturnValue = "Sphere";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane:
        functionReturnValue = "Best Fit Plane";
        break;
    case SolidWorks.Interop.swdimxpert.swDimXpertFeatureType_e.swDimXpertFeature_Surface:
        functionReturnValue = "Surface";
        break;
    default:
        functionReturnValue = "<unknown> " + featureTypeIndex;
        break;
}
return functionReturnValue;
}
}

```

DimXpert Main Module Example (C#)

```
// This module is a component of
// Get DimXpert Features and Annotations in a Model Example \(C#\).

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdimxpert;
using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Collections.ObjectModel;
namespace DimXpert_text_v2_cs.csproj
{
    partial class SolidWorksMacro
    {
        //This application allows the user to view all of the DimXpert feature and annotation objects in a model.
        public void Main()
        {
            ModelDoc2 swModelDoc = default(ModelDoc2);
            swModelDoc = (ModelDoc2)swApp.ActiveDoc;
            if (swModelDoc == null)
            {
                return;
            }
            Scripting.FileSystemObject f = new Scripting.FileSystemObject();
            Scripting.TextStream textStr = default(Scripting.TextStream);
            textStr = f.CreateTextFile("C:\\temp\\dimXpertInfo.txt", true, false);
            if (textStr == null)
            {
                Debug.Print("Error creating temp file.");
                return;
            }
            log("Starting DimXpert log...", textStr);
            retrieve_info_text(swApp, textStr);
            textStr.Close();
        }
        private void log(string text, Scripting.TextStream textStr)
        {
            Debug.Print(text);
            textStr.WriteLine(text);
        }
        private void retrieve_info_text(SldWorks swapp, Scripting.TextStream textStr)
        {
            DimXpertManager dimXpertMgr = default(DimXpertManager);
            ModelDocExtension modelDocExtension = default(ModelDocExtension);
            modelDocExtension = swapp.IActiveDoc2.Extension;
            dimXpertMgr = modelDocExtension.get_DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
true);
            log("Model: " + swapp.IActiveDoc2.GetPathName(), textStr);
            DimXpertPart dimXpertPartObj = default(DimXpertPart);
            dimXpertPartObj = (DimXpertPart)dimXpertMgr.DimXpertPart;
            SolidWorks.Interop.swdimxpert.DimXpertPart dimXpertPart = default(SolidWorks.Interop.swdimxpert.DimXpertPart);
            dimXpertPart = dimXpertPartObj;
            object[] vFeatures = null;
            object[] vAnnotations = null;
            vFeatures = (object[])dimXpertPart.GetFeatures();
            vAnnotations = (object[])dimXpertPart.GetAnnotations();
        }
    }
}
```

```

log("-----", textStr);
log("Features...", textStr);
log("-----", textStr);
DimXpertFeature featTemp = default(DimXpertFeature);
long featureIndex = 0;
for (featureIndex = 0; featureIndex <= vFeatures.GetUpperBound(0); featureIndex++)
{
    featTemp = (DimXpertFeature)vFeatures[featureIndex];
    DimXpertFeatureData FeatureData = new DimXpertFeatureData();
    Collection<string> featureDataText = default(Collection<string>);
    featureDataText = FeatureData.FeatureData(featTemp);
    int featureTextIndex = 0;
    log("DimXpertFeature...", textStr);
    for (featureTextIndex = 0; featureTextIndex <= featureDataText.Count - 1; featureTextIndex++)
    {
        log(featureDataText[featureTextIndex], textStr);
    }
    log(" ", textStr);
    log(" ", textStr);
}
log("-----", textStr);
log("Annotations...", textStr);
log("-----", textStr);
DimXpertAnnotation annotationTemp = default(DimXpertAnnotation);
long annotationIndex = 0;
for (annotationIndex = 0; annotationIndex <= vAnnotations.GetUpperBound(0); annotationIndex++)
{
    annotationTemp = (DimXpertAnnotation)vAnnotations[annotationIndex];
    Debug.Print(annotationTemp.Name);
    DimXpertAnnotationData AnnotationData = new DimXpertAnnotationData();
    Collection<string> AnnotationDataText = default(Collection<string>);
    AnnotationDataText = AnnotationData.AnnotationData(annotationTemp, dimXpertPart);
    int annotationTextIndex = 0;
    log("DimXpertAnnotation...", textStr);
    for (annotationTextIndex = 0; annotationTextIndex <= AnnotationDataText.Count - 1; annotationTextIndex++)
    {
        log(AnnotationDataText[annotationTextIndex], textStr);
    }
    log(" ", textStr);
    log(" ", textStr);
}
log("-----", textStr);
log("Block Tolerances...", textStr);
log("-----", textStr);
listBlockTolerances_text(dimXpertPart, textStr);
}
private void listBlockTolerances_text(SolidWorks.Interop.swdimxpert.DimXpertPart dimXpertPart, Scripting.TextStream
textStr)
{
    SolidWorks.Interop.swdimxpert.DimXpertBlockTolerances blockTols =
default(SolidWorks.Interop.swdimxpert.DimXpertBlockTolerances);
    bool boolstatus = false;
    double lin1 = 0;
    int lin1prec = 0;
    double lin2 = 0;
    int lin2prec = 0;
    double lin3 = 0;
    int lin3prec = 0;
    double ang = 0;
    SolidWorks.Interop.swdimxpert.swDimxpertISO2768PartType_e isoCode =
default(SolidWorks.Interop.swdimxpert.swDimxpertISO2768PartType_e);
    blockTols = dimXpertPart.GetBlockTolerances();
    if ((blockTols != null))
    {

```

```

        switch (blockTols.Type)
        {
            case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch:
                boolstatus = blockTols.GetToleranceValues(ref lin1, ref lin1prec, ref lin2, ref lin2prec, ref lin3, ref
lin3prec, ref ang);
                log("swDimXpertBlockToleranceType_ASMEInch", textStr);
                log("Linear1: " + System.String.Format("{0:D}", lin1prec.ToString()) +
                    " Places = " + System.String.Format("{0:D}", lin1.ToString()) + " " +
                    "Linear2: " + System.String.Format("{0:D}", lin2prec.ToString()) +
                    " Places = " + System.String.Format("{0:D}", lin2.ToString()) + " " +
                    "Linear3: " + System.String.Format("{0:D}", lin3prec.ToString()) +
                    " Places = " + System.String.Format("{0:D}", lin3.ToString()) + " " +
                    "Angular = " + System.String.Format("{0:D}", (ang * 57.2957795130823).ToString()), textStr);
                break;
            case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768:
                log("swDimXpertBlockToleranceType_ISO2768", textStr);
                boolstatus = blockTols.GetISO2768PartType(ref isoCode);
                switch (isoCode)
                {
                    case swDimXpertISO2768PartType_e.swDimXpertISO2768PartType_Fine:
                        log("General Tolerance: Fine", textStr);
                        break;
                    case swDimXpertISO2768PartType_e.swDimXpertISO2768PartType_Medium:
                        log("General Tolerance: Medium", textStr);
                        break;
                    case swDimXpertISO2768PartType_e.swDimXpertISO2768PartType_Coarse:
                        log("General Tolerance: Coarse", textStr);
                        break;
                    case swDimXpertISO2768PartType_e.swDimXpertISO2768PartType_VeryCoarse:
                        log("General Tolerance: Very Coarse", textStr);
                        break;
                }
                break;
        }
    }
}

public SldWorks swApp;
}
}

```

Insert Basic Dimension Example (C#)

This example shows how to insert a DimXpert distance-between dimension.

```
//-----
// Preconditions:
// 1. Ensure that the part to open exists.
// 2. Reference the SOLIDWORKS DimXpert interop assembly:
//    a. Right-click the project in Project Explorer.
//    b. Click Add Reference.
//    c. Click the Browse tab.
//    d. Select install_dir\api\redist\swdimxpert.dll.
//
// Postconditions: Inspect the graphics view and the DimXpertManager pane.
//
// NOTE: Because the part is used elsewhere, do not save any changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdimxpert;
using System.Runtime.InteropServices;
namespace InsertBasicDimension_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Configuration swConfig;
        DimXpertManager swDimXPertMgr;
        DimXpertPart swDimXPertPart;
        DimXpertDimensionOption swDimXPertDimOpt;
        object[] dblDimXPertTextPos = new object[3];
        object[] varDimXPertTextPos;
        bool boolstatus;
        int status;
        int warnings;

        public void Main()
        {
            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2017\\Simulation
Examples\\actuator_casing.sldprt", 1, 0, "", ref status, ref warnings);
            swApp.ActivateDoc2("actuator_casing.sldprt", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            Part.ShowNamedView2("Isometric", 7);

            swConfig = Part.ConfigurationManager.ActiveConfiguration;
            swDimXPertMgr = Part.Extension.get_DimXpertManager(swConfig.Name, true);
            swDimXPertPart = (DimXpertPart)swDimXPertMgr.DimXpertPart;
            swDimXPertDimOpt = swDimXPertPart.GetDimOption();
            boolstatus = Part.Extension.SelectByID2("", "FACE", 0.0832566935649766, 0.0141096473029049, 0.0143449005094567, false, 0,
            boolstatus = Part.Extension.SelectByID2("", "FACE", 0.0751728497303361, 0.000895850622406916, 0.0190499999998792, true, 0,
            dblDimXPertTextPos[0] = 0.083208;
            dblDimXPertTextPos[1] = 0.045464;
            dblDimXPertTextPos[2] = 0.019052;
            varDimXPertTextPos = dblDimXPertTextPos;
            swDimXPertDimOpt.TextPosition = varDimXPertTextPos;
            boolstatus = swDimXPertPart.InsertBasicDimension(swDimXPertDimOpt);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Insert Basic Dimension Example (VBA)

This example shows how to insert a DimXpert distance-between dimension.

```

'-----
' Preconditions:
' 1. Ensure that the part to open exists.
' 2. Reference SOLIDWORKS version DimXpert type library:
'    a. Click Tools > References > Browse.
'    b. Select install_dir\swdimxpert.tlb.
'    c. Click Open.
'
' Postconditions: Inspect the graphics view and the DimXpertManager pane.
'
' NOTE: Because the part is used elsewhere, do not save any changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim swConfig As SldWorks.Configuration
Dim swDimXPertMgr As SldWorks.DimXpertManager
Dim swDimXPertPart As SwDimXpert.DimXpertPart
Dim swDimXPertDimOpt As SwDimXpert.DimXpertDimensionOption
Dim dblDimXPertTextPos(2) As Double
Dim varDimXPertTextPos As Variant
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long
Option Explicit
Sub main()

    Set swApp = Application.SldWorks

    Set Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\Simulation Examples\actuator_casing.sldprt", 1,
0, "", longstatus, longwarnings)
    swApp.ActivateDoc2 "actuator_casing.sldprt", False, longstatus
    Set Part = swApp.ActiveDoc

    Part.ShowNamedView2 "Isometric", 7

    Set swConfig = Part.ConfigurationManager.ActiveConfiguration
    Set swDimXPertMgr = Part.Extension.DimXpertManager(swConfig.Name, True)
    Set swDimXPertPart = swDimXPertMgr.DimXpertPart
    Set swDimXPertDimOpt = swDimXPertPart.GetDimOption
    boolstatus = Part.Extension.SelectByID2("", "FACE", 8.32566935649766E-02, 1.41096473029049E-02, 1.43449005094567E-02, False, 0,
Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("", "FACE", 7.51728497303361E-02, 8.95850622406916E-04, 1.90499999998792E-02, True, 0,
Nothing, 0)
    dblDimXPertTextPos(0) = 0.083208
    dblDimXPertTextPos(1) = 0.045464
    dblDimXPertTextPos(2) = 0.019052
    varDimXPertTextPos = dblDimXPertTextPos
    swDimXPertDimOpt.TextPosition = varDimXPertTextPos
    boolstatus = swDimXPertPart.InsertBasicDimension(swDimXPertDimOpt)

End Sub

```

Insert Basic Dimension Example (VB.NET)

This example shows how to insert a DimXpert distance-between dimension.

```
'-----
' Preconditions:
' 1. Ensure that the part to open exists.
' 2. Reference the SOLIDWORKS DimXpert interop assembly:
'    a. Right-click the project in Project Explorer.
'    b. Click Add Reference.
'    c. Click the Browse tab.
'    d. Select install_dir\api\redist\swdimxpert.dll.
'
' Postconditions: Inspect the graphics view and the DimXpertManager pane.
'
' NOTE: Because the part is used elsewhere, do not save any changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdimxpert
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim swConfig As Configuration
    Dim swDimXPertMgr As DimXpertManager
    Dim swDimXPertPart As DimXpertPart
    Dim swDimXPertDimOpt As DimXpertDimensionOption
    Dim dblDimXPertTextPos(2) As Double
    Dim varDimXPertTextPos As Object
    Dim boolstatus As Boolean
    Dim status As Integer, warnings As Integer
```

```
    Sub main()
```

```
        Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\Simulation
Examples\actuator_casing.sldprt", 1, 0, "", status, warnings)
        swApp.ActivateDoc2("actuator_casing.sldprt", False, longstatus)
        Part = swApp.ActiveDoc

        Part.ShowNamedView2("Isometric", 7)

        swConfig = Part.ConfigurationManager.ActiveConfiguration
        swDimXPertMgr = Part.Extension.DimXpertManager(swConfig.Name, True)
        swDimXPertPart = swDimXPertMgr.DimXpertPart
        swDimXPertDimOpt = swDimXPertPart.GetDimOption
        boolstatus = Part.Extension.SelectByID("", "FACE", 0.0832566935649766, 0.0141096473029049, 0.0143449005094567, False, 0, No
        boolstatus = Part.Extension.SelectByID("", "FACE", 0.0751728497303361, 0.000895850622406916, 0.0190499999998792, True, 0, No
        dblDimXPertTextPos(0) = 0.083208
        dblDimXPertTextPos(1) = 0.045464
        dblDimXPertTextPos(2) = 0.019052
        varDimXPertTextPos = dblDimXPertTextPos
        swDimXPertDimOpt.TextPosition = varDimXPertTextPos
        boolstatus = swDimXPertPart.InsertBasicDimension(swDimXPertDimOpt)
```

```
    End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Get and Set Datum Example (C#)

This example shows how to get and set DimXpert datum annotations.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\dimxpert\bracket_auto_manual.sldprt.
// 2. Select a face.
// 3. Open an Immediate window.
// 4. Ensure that the SOLIDWORKS DimXpert interop assembly is referenced:
//    a. Right-click the project in Project Explorer.
//    b. Select Add Reference.
//    c. Click the Browse tab.
//    d. Find and select install_dir\api\redist\swdimxpert.dll.
//
// Postconditions:
// 1. Inspect the Immediate window.
// 2. Observe Plane1 and datum G on the DimXpertManager tab.
//
// NOTE: Because this part is elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdimxpert;
using System;
using System.Diagnostics;
namespace InsertDatum_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        DimXpertPart dimXpertPart;

        public void Main()
        {
            ModelDoc2 swModelDoc = default(ModelDoc2);
            swModelDoc = (ModelDoc2)swApp.ActiveDoc;

            if (swModelDoc == null)
            {
                return;
            }

            DimXpertManager dimXpertMgr = default(DimXpertManager);
            dimXpertMgr = swApp.IActiveDoc2.Extension.get_DimXpertManager(swApp.IActiveDoc2.IGetActiveConfiguration().Name, true);
            Debug.Print("Model: " + swApp.IActiveDoc2.GetPathName());

            DimXpertPart dimXpertPartObj = default(DimXpertPart);

            dimXpertPartObj = (DimXpertPart)dimXpertMgr.DimXpertPart;
            dimXpertPart = dimXpertPartObj;

            DimXpertDimensionOption dimOption = default(DimXpertDimensionOption);
            dimOption = dimXpertPart.GetDimOption();
            dimOption.DatumLength = 0.06;
            int[] dimarray = new int[1];
            dimarray[0] = 0;
            object dimvar = null;
            dimvar = dimarray;
            dimOption.FeatureSelectorOptions = dimvar;

            // Insert datum
            dimXpertPart.InsertDatum(dimOption);

            object[] vAnnotations = null;
            vAnnotations = (object[])dimXpertPart.GetAnnotations();

            Debug.Print("-----");
            Debug.Print("Annotations...");
            Debug.Print("-----");

            DimXpertAnnotation annotationTemp = default(DimXpertAnnotation);
            int annotationIndex = 0;
            for (annotationIndex = 0; annotationIndex <= vAnnotations.GetUpperBound(0); annotationIndex++)
            {
                annotationTemp = (DimXpertAnnotation)vAnnotations[annotationIndex];
                AnnotationData(annotationTemp);
            }
        }

        public void AnnotationData(DimXpertAnnotation annotation)
        {
            swDimXpertAnnotationType_e annoType = 0;

            //general info
```

```

        GeneralInfo(annotation);
        annoType = annotation.Type;

        if (annoType == swDimXpertAnnotationType_e.swDimXpertDatum)
        {
            DatumData((DimXpertDatum)annotation);
        }
    }

private void GeneralInfo(DimXpertAnnotation annotation)
{
    string annoType = null;
    object modelObj = null;
    Feature modelFeature = default(Feature);

    Debug.Print("");
    Debug.Print("Name: " + annotation.Name);

    annoType = annotation.TypeNameFromObject(annotation);

    Debug.Print("Type: " + annoType);
    Debug.Print("Display Entity: " + DisplayEntity(annotation));

    modelObj = annotation.GetModelFeature();
    modelFeature = (Feature)modelObj;

    if ((modelFeature != null))
    {
        Debug.Print("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2() + ")");
    }
}

private void DatumData(DimXpertDatum annotation)
{
    // the datum letter
    Debug.Print("");

    Debug.Print("Datum Letter: " + annotation.Identifier);
}

private string annotationTypeNameFromObject(DimXpertAnnotation anno)
{
    return annotation.TypeNameFromTypeNumber(anno.Type);
}

private string annotationTypeNameFromTypeNumber(swDimXpertAnnotationType_e annoTypeIndex)
{
    string functionReturnValue = null;
    switch (annoTypeIndex)
    {
        case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween:
            functionReturnValue = "DistanceBetween Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore:
            functionReturnValue = "CounterBore Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_Depth:
            functionReturnValue = "Depth Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter:
            functionReturnValue = "CounterSinkDiameter Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension:
            functionReturnValue = "ChamferDimension Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween:
            functionReturnValue = "AngleBetween Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle:
            functionReturnValue = "CounterSinkAngle Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle:
            functionReturnValue = "ConeAngle Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter:
            functionReturnValue = "Diameter Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_Length:
            functionReturnValue = "Length Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_Radius:
            functionReturnValue = "Radius Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_Width:
            functionReturnValue = "Width Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween:
            functionReturnValue = "CompositeDistanceBetween Dim";
    }
}

```

```

        break;
    case swDimXpertAnnotationType_e.swDimXpertDatum:
        functionReturnValue = "Datum";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Position:
        functionReturnValue = "Position Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition:
        functionReturnValue = "CompositePosition Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry:
        functionReturnValue = "Symmetry Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity:
        functionReturnValue = "Concentricity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile:
        functionReturnValue = "LineProfile Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile:
        functionReturnValue = "CompositeLineProfile Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile:
        functionReturnValue = "SurfaceProfile Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile:
        functionReturnValue = "CompositeSurfaceProfile Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity:
        functionReturnValue = "Angularity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism:
        functionReturnValue = "Parallelism Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity:
        functionReturnValue = "Perpendicularity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout:
        functionReturnValue = "TotalRunout Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout:
        functionReturnValue = "CircularRunout Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness:
        functionReturnValue = "Flatness Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity:
        functionReturnValue = "Circularity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity:
        functionReturnValue = "Cylindricity Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness:
        functionReturnValue = "Straightness Tol";
        break;
    case swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency:
        functionReturnValue = "Tangency Tol";
        break;
    default:
        functionReturnValue = "<unknown> " + annoTypeIndex.ToString();
        break;
    }

    return functionReturnValue;
}
private string DisplayEntity(DimXpertAnnotation annotation)
{
    string str = null;
    object dispEnt = null;
    Annotation swAnnot = default(Annotation);
    str = null;
    dispEnt = annotation.GetDisplayEntity();
    if ((dispEnt != null))
    {
        if (dispEnt is Annotation)
        {
            swAnnot = (Annotation)dispEnt;
            str = swAnnot.GetName();
        }
    }
    return str;
}

public SldWorks swApp;
}
}

```

Get and Set Datum Example (VBA)

This example shows how to get and set DimXpert datum annotations.

```

'-----
' Preconditions:
' 1. Open: public_documents\samples\tutorial\dimxpert\bracket_auto_manual.sldprt.
' 2. Select a face.
' 3. Open an Immediate window.
' 4. Ensure that the latest SOLIDWORKS DimXpert type library is loaded:
'    a. click Tools > References.
'    b. Click Browse.
'    c. Find and select install_dir\swdimxpert.tlb.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. Observe Plane1 and Datum G on the DimXpertManager tab.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----

Option Explicit
Dim dimXpertPart As dimXpertPart
Sub Main()
    Dim swapp As SldWorks.SldWorks
    Set swapp = Application.SldWorks
    Dim swModelDoc As SldWorks.ModelDoc2
    Set swModelDoc = swapp.ActiveDoc

    If swModelDoc Is Nothing Then
        Exit Sub
    End If

    Dim dimXpertMgr As SldWorks.DimXpertManager
    Set dimXpertMgr = swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name, True)
    Debug.Print "Model: " & swapp.IActiveDoc2.GetPathName

    Dim dimXpertPartObj As dimXpertPart
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart
    Set dimXpertPart = dimXpertPartObj

    Dim dimOption As DimXpertDimensionOption
    Set dimOption = dimXpertPart.GetDimOption
    dimOption.DatumLength = 0.06
    Dim dimarray(0) As Long
    dimarray(0) = 0
    Dim dimvar As Variant
    dimvar = dimarray
    dimOption.FeatureSelectorOptions = dimvar
    ' Insert datum
    dimXpertPart.InsertDatum dimOption

    Dim vAnnotations As Variant
    vAnnotations = dimXpertPart.GetAnnotations()
    Debug.Print "-----"
    Debug.Print "Annotations..."
    Debug.Print "-----"

    Dim annotationTemp As DimXpertAnnotation
    Dim annotationIndex As Long
    For annotationIndex = 0 To UBound(vAnnotations)
        Set annotationTemp = vAnnotations(annotationIndex)
        Call AnnotationData(annotationTemp)
    Next

End Sub
Public Sub AnnotationData(annotation As DimXpertAnnotation)
    Dim annoType As Long
    'general info
    Call GeneralInfo(annotation)
    annoType = annotation.Type
    If annoType = swDimXpertDatum Then
        Call DatumData(annotation)
    End If
End Sub
Private Sub GeneralInfo(annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Object
    Dim modelFeature As SldWorks.Feature
    Debug.Print ""
    Debug.Print ("Name: " + annotation.Name)
    annoType = annotationTypeNameFromObject(annotation)

```

```

Debug.Print ("Type: " + annoType)
Debug.Print ("Display Entity: " + DisplayEntity(annotation))

Set modelObj = annotation.GetModelFeature
Set modelFeature = modelObj

If Not (modelFeature Is Nothing) Then
    Debug.Print ("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2() + ")")
End If

End Sub

Private Sub DatumData(annotation As DimXpertDatum)
    ' the datum letter
    Debug.Print ("")
    Debug.Print ("Datum Letter: " + annotation.Identifier)

End Sub

Private Function annotationTypeNameFromObject(anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function

Private Function annotationTypeNameFromTypeNumber(annoTypeIndex As Long) As String
    Select Case annoTypeIndex
        Case swDimXpertDimTol_DistanceBetween
            annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
        Case swDimXpertDimTol_CounterBore
            annotationTypeNameFromTypeNumber = "CounterBore Dim"
        Case swDimXpertDimTol_Depth
            annotationTypeNameFromTypeNumber = "Depth Dim"
        Case swDimXpertDimTol_CounterSinkDiameter
            annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
        Case swDimXpertDimTol_ChamferDimension
            annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
        Case swDimXpertDimTol_AngleBetween
            annotationTypeNameFromTypeNumber = "AngleBetween Dim"
        Case swDimXpertDimTol_CounterSinkAngle
            annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
        Case swDimXpertDimTol_ConeAngle
            annotationTypeNameFromTypeNumber = "ConeAngle Dim"
        Case swDimXpertDimTol_Diameter
            annotationTypeNameFromTypeNumber = "Diameter Dim"
        Case swDimXpertDimTol_Length
            annotationTypeNameFromTypeNumber = "Length Dim"
        Case swDimXpertDimTol_Radius
            annotationTypeNameFromTypeNumber = "Radius Dim"
        Case swDimXpertDimTol_Width
            annotationTypeNameFromTypeNumber = "Width Dim"
        Case swDimXpertDimTol_CompositeDistanceBetween
            annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
        Case swDimXpertDatum
            annotationTypeNameFromTypeNumber = "Datum"
        Case swDimXpertGeoTol_Position
            annotationTypeNameFromTypeNumber = "Position Tol"
        Case swDimXpertGeoTol_CompositePosition
            annotationTypeNameFromTypeNumber = "CompositePosition Tol"
        Case swDimXpertGeoTol_Symmetry
            annotationTypeNameFromTypeNumber = "Symmetry Tol"
        Case swDimXpertGeoTol_Concentricity
            annotationTypeNameFromTypeNumber = "Concentricity Tol"
        Case swDimXpertGeoTol_LineProfile
            annotationTypeNameFromTypeNumber = "LineProfile Tol"
        Case swDimXpertGeoTol_CompositeLineProfile
            annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
        Case swDimXpertGeoTol_SurfaceProfile
            annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
        Case swDimXpertGeoTol_CompositeSurfaceProfile
            annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
        Case swDimXpertGeoTol_Angularity
            annotationTypeNameFromTypeNumber = "Angularity Tol"
        Case swDimXpertGeoTol_Parallelism
            annotationTypeNameFromTypeNumber = "Parallelism Tol"
        Case swDimXpertGeoTol_Perpendicularity
            annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
        Case swDimXpertGeoTol_TotalRunout
            annotationTypeNameFromTypeNumber = "TotalRunout Tol"
        Case swDimXpertGeoTol_CircularRunout
            annotationTypeNameFromTypeNumber = "CircularRunout Tol"
        Case swDimXpertGeoTol_Flatness
            annotationTypeNameFromTypeNumber = "Flatness Tol"
        Case swDimXpertGeoTol_Circularity
            annotationTypeNameFromTypeNumber = "Circularity Tol"
        Case swDimXpertGeoTol_Cylindricity
            annotationTypeNameFromTypeNumber = "Cylindricity Tol"
        Case swDimXpertGeoTol_Straightness
            annotationTypeNameFromTypeNumber = "Straightness Tol"
        Case swDimXpertGeoTol_Tangency
            annotationTypeNameFromTypeNumber = "Tangency Tol"
        Case Else
            annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)
    End Select

    ' returns a string containing the names of the SW display entities
End Function

```

```
Private Function DisplayEntity(annotation As DimXpertAnnotation) As String
    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As SldWorks.annotation
    Set dispEnt = annotation.GetDisplayEntity
    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is SldWorks.annotation Then
            Set swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If
    DisplayEntity = str
End Function
```

Get and Set Datum Example (VB.NET)

This example shows how to get and set DimXpert datum annotations.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\dimxpert\bracket_auto_manual.sldprt.
' 2. Select a face.
' 3. Open an Immediate window.
' 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:
'   a. Right-click the project in Project Explorer.
'   b. Select Add Reference.
'   c. Click the Browse tab.
'   d. Find and select install_dir\api\redist\swdimxpert.dll.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. Observe Plane1 and datum G on the DimXpertManager tab.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdimxpert
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim dimXpertPart As dimXpertPart

    Sub Main()

        Dim swModelDoc As ModelDoc2
        swModelDoc = swapp.ActiveDoc

        If swModelDoc Is Nothing Then
            Exit Sub
        End If

        Dim dimXpertMgr As DimXpertManager
        dimXpertMgr = swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name, True)
        Debug.Print("Model: " & swapp.IActiveDoc2.GetPathName)

        Dim dimXpertPartObj As dimXpertPart

        dimXpertPartObj = dimXpertMgr.dimXpertPart
        dimXpertPart = dimXpertPartObj

        Dim dimOption As DimXpertDimensionOption
        dimOption = dimXpertPart.GetDimOption
        dimOption.DatumLength = 0.06
        Dim dimarray(0) As Long
        dimarray(0) = 0
        Dim dimvar As Object
        dimvar = dimarray
        dimOption.FeatureSelectorOptions = dimvar

        ' Insert datum
        dimXpertPart.InsertDatum(dimOption)

        Dim vAnnotations As Object
        vAnnotations = dimXpertPart.GetAnnotations()

        Debug.Print("-----")
        Debug.Print("Annotations...")
        Debug.Print("-----")

        Dim annotationTemp As DimXpertAnnotation
        Dim annotationIndex As Long
        For annotationIndex = 0 To UBound(vAnnotations)
            annotationTemp = vAnnotations(annotationIndex)
            Call AnnotationData(annotationTemp)
        Next

    End Sub

    Public Sub AnnotationData(ByVal annotation As DimXpertAnnotation)

        Dim annoType As Long

        'general info
        Call GeneralInfo(annotation)
        annoType = annotation.Type

        If annoType = swDimXpertAnnotationType_e.swDimXpertDatum Then
            Call DatumData(annotation)
        End If
    End Sub
End Class

```

End Sub

Private Sub GeneralInfo(ByVal annotation As DimXpertAnnotation)

```
Dim annoType As String
Dim modelObj As Object
Dim modelFeature As Feature
```

```
Debug.Print("")
Debug.Print("Name: " + annotation.Name)
```

```
annoType = annotation.TypeNameFromObject(annotation)
```

```
Debug.Print("Type: " + annoType)
Debug.Print("Display Entity: " + DisplayEntity(annotation))
```

```
modelObj = annotation.GetModelFeature
modelFeature = modelObj
```

```
If Not (modelFeature Is Nothing) Then
    Debug.Print("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2() + ")")
End If
```

End Sub

Private Sub DatumData(ByVal annotation As DimXpertDatum)

```
' the datum letter
Debug.Print("")
Debug.Print("Datum Letter: " + annotation.Identifier)
```

End Sub

Private Function annotationTypeNameFromObject(ByVal anno As DimXpertAnnotation) As String

```
annotationTypeNameFromObject = annotation.TypeNameFromTypeNumber(anno.Type)
```

End Function

Private Function annotationTypeNameFromTypeNumber(ByVal annoTypeIndex As Long) As String

```
Select Case annoTypeIndex
```

```
Case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween
    annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore
    annotationTypeNameFromTypeNumber = "CounterBore Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Depth
    annotationTypeNameFromTypeNumber = "Depth Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
    annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension
    annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween
    annotationTypeNameFromTypeNumber = "AngleBetween Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle
    annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle
    annotationTypeNameFromTypeNumber = "ConeAngle Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter
    annotationTypeNameFromTypeNumber = "Diameter Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Length
    annotationTypeNameFromTypeNumber = "Length Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Radius
    annotationTypeNameFromTypeNumber = "Radius Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Width
    annotationTypeNameFromTypeNumber = "Width Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween
    annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"

Case swDimXpertAnnotationType_e.swDimXpertDatum
    annotationTypeNameFromTypeNumber = "Datum"

Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Position
    annotationTypeNameFromTypeNumber = "Position Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition
    annotationTypeNameFromTypeNumber = "CompositePosition Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry
    annotationTypeNameFromTypeNumber = "Symmetry Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity
    annotationTypeNameFromTypeNumber = "Concentricity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile
    annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
    annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile
    annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
    annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity
    annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism
    annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity
    annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout
```

```

        annotationTypeNameFromTypeNumber = "TotalRunout Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout
        annotationTypeNameFromTypeNumber = "CircularRunout Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness
        annotationTypeNameFromTypeNumber = "Flatness Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity
        annotationTypeNameFromTypeNumber = "Circularity Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity
        annotationTypeNameFromTypeNumber = "Cylindricity Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness
        annotationTypeNameFromTypeNumber = "Straightness Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency
        annotationTypeNameFromTypeNumber = "Tangency Tol"
    Case Else
        annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)

```

```
End Select
```

```
End Function
```

```
Private Function DisplayEntity(ByVal annotation As DimXpertAnnotation) As String
```

```

    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As Annotation
    str = Nothing
    dispEnt = annotation.GetDisplayEntity
    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is Annotation Then
            swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If
    DisplayEntity = str
End Function

```

```
Public swApp As SldWorks
```

```
End Class
```

Get and Set Location Dimension Example (C#)

This example shows how to get and set DimXpert distance-between or angle-between dimensions.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\dimxpert\bracket_ads_plusminus.sldprt.
// 2. Multi-select two faces that:
//    a. share a common edge to get the angle-between dimension.
//    b. do not share a common edge to get the distance-between dimension.
// 3. Open an Immediate window.
// 4. Ensure that the latest SOLIDWORKS DimXpert interop assembly is referenced:
//    a. Right-click the project in Project Explorer.
//    b. Select Add Reference.
//    c. Click the Browse tab.
//    d. Find and select install_dir\api\redist\swdimxpert.dll.
//
// Postconditions:
// 1. Inspect the Immediate window.
// 2. Observe the DistanceBetween1 or AngleBetween1 annotation on
//    the DimXpertManager tab.
//
// NOTE: Because this part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdimxpert;
using System;
using System.Diagnostics;

namespace InsertLocationDimension_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        DimXpertPart dimXpertPart;
        public void Main()
        {
            ModelDoc2 swModelDoc = default(ModelDoc2);
            swModelDoc = (ModelDoc2)swApp.ActiveDoc;
            swDimXpertAnnotationType_e annoType = 0;

            if (swModelDoc == null)
            {
                return;
            }

            DimXpertManager dimXpertMgr = default(DimXpertManager);
            dimXpertMgr = swApp.IActiveDoc2.Extension.get_DimXpertManager(swApp.IActiveDoc2.IGetActiveConfiguration().Name, true);
            Debug.Print("Model: " + swApp.IActiveDoc2.GetPathName());

            DimXpertPart dimXpertPartObj = default(DimXpertPart);
            dimXpertPartObj = (DimXpertPart)dimXpertMgr.DimXpertPart;
            dimXpertPart = dimXpertPartObj;

            DimXpertDimensionOption dimOption = default(DimXpertDimensionOption);
            dimOption = dimXpertPart.GetDimOption();

            // The following line applies to linear location dimensions only.
            dimOption.DimensionPositionOption = 0; // Specify a dimension aint the normal to the face

            // Insert the dimension
            dimXpertPart.InsertLocationDimension(dimOption);

            object[] vAnnotations = null;
            vAnnotations = (object[])dimXpertPart.GetAnnotations();

            Debug.Print("-----");
            Debug.Print("Annotations...");
            Debug.Print("-----");

            DimXpertAnnotation annotationTemp = default(DimXpertAnnotation);
            int annotationIndex = 0;
            for (annotationIndex = 0; annotationIndex <= vAnnotations.GetUpperBound(0); annotationIndex++)
            {
                annotationTemp = (DimXpertAnnotation)vAnnotations[annotationIndex];
                GeneralInfo(annotationTemp);
                annoType = annotationTemp.Type;
                DimensionToleranceData((DimXpertDimensionTolerance)annotationTemp);
            }
        }

        private void GeneralInfo(DimXpertAnnotation annotation)
        {
            string annoType = null;
            object modelObj = null;
            Feature modelFeature = default(Feature);
        }
    }
}
```

```

Debug.Print("");
Debug.Print("Name: " + annotation.Name);

annoType = annotation.TypeNameFromObject(annotation);

Debug.Print("Type: " + annoType);
Debug.Print("Display Entity: " + DisplayEntity(annotation));

modelObj = annotation.GetModelFeature();
modelFeature = (Feature)modelObj;
if ((modelFeature != null))
{
    Debug.Print("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2() + ")");
}
}

private void DimensionToleranceData(DimXpertDimensionTolerance annotation)
{
    bool isAngleType = false;
    swDimXpertAnnotationType_e annoType = default(swDimXpertAnnotationType_e);

    double upper = 0;
    double lower = 0;
    double plus = 0;
    double minus = 0;
    bool boolstatus = false;

    DimXpertAnnotation anno = (DimXpertAnnotation)annotation;
    annoType = anno.Type;
    isAngleType = false;

    Debug.Print(annoType.ToString());

    Debug.Print("");
    Debug.Print("Dimension Tolerance Compartment");

    if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween)
    {
        DimXpertDistanceBetweenDimTol distancebetween = default(DimXpertDistanceBetweenDimTol);
        distancebetween = (DimXpertDistanceBetweenDimTol)annotation;
        DistanceBetweenData(distancebetween);
    }
    else if (annoType == swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween)
    {
        isAngleType = true;
        IDimXpertAngleBetweenDimTol angleBetween = default(IDimXpertAngleBetweenDimTol);
        angleBetween = (DimXpertAngleBetweenDimTol)annotation;

        // the origin and tolerance feature
        Debug.Print("Origin Feature: " + angleBetween.OriginFeature.Name);

        // is supplement angle
        Debug.Print("Supplement Angle: " + (angleBetween.Supplement ? "True" : "False"));
    }

    // conversion from radians to degrees when dimension type is angle

    double dbl = 0;
    if (isAngleType)
    {
        dbl = 57.2957795130823;
    }
    else
    {
        dbl = 1.0;
    }

    // the nominal, and upper and lower limits of size of the dimension

    Debug.Print("");
    Debug.Print("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue() * dbl));
    boolstatus = annotation.GetUpperAndLowerLimit(ref upper, ref lower);
    Debug.Print("Get Upper Limit: " + FormatDouble(upper * dbl));
    Debug.Print("Get Lower Limit: " + FormatDouble(lower * dbl));

    // the upper and lower tolerance value by type
    switch (annotation.DimensionType)
    {
        case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockTolerance:
            Debug.Print("Dimension Type: Block Tolerance");
            break;
        // block tolerance
        case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal:
            DimXpertBlockTolerances blockTols = default(DimXpertBlockTolerances);
            blockTols = dimXpertPart.GetBlockTolerances();
            switch (blockTols.Type)
            {
                case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch:
                    Debug.Print("Dimension Type: Block Tolerance No Nominal");
                    if (isAngleType)
                    {
                        Debug.Print("Angular Block Tolerance");
                    }
            }
    }
}

```

```

        else
        {
            Debug.Print("Block Tolerance Decimal Places: " +
(annotation.BlockToleranceDecimalPlaces.ToString()));
        }
        break;
        case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768:
            Debug.Print("Dimension Type: General Tolerance");
            break;
    }
    break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFits:
        Debug.Print("Dimension Type: Limits and Fits");
        break;
    // limits and fits tolerance
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFitsNoNominal:
        Debug.Print("Dimension Type: Limits and Fits No Nominal");
        Debug.Print("Limits and Fits: " + annotation.LimitsAndFitsCode);
        break;
    // limit dimension
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_LimitDimension:
        Debug.Print("Dimension Type: Limit Dimension");
        boolstatus = annotation.GetUpperAndLowerLimit(ref upper, ref lower);
        Debug.Print("Get Upper Limit: " + FormatDouble(upper * dbl));
        Debug.Print("Get Lower Limit: " + FormatDouble(lower * dbl));
        break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MAXTolerance:
        Debug.Print("Dimension Type: MAXTolerance");
        break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MINTolerance:
        Debug.Print("Dimension Type: MINTolerance");
        break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_NoTolerance:
        Debug.Print("Dimension Type: NoTolerance");
        break;
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusDimension:
        Debug.Print("Dimension Type: Plus Minus Dimension");
        break;
    // plus and minus dimension
    case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal:
        Debug.Print("Dimension Type: Plus Minus No Nominal");
        boolstatus = annotation.GetPlusAndMinusTolerance(ref plus, ref minus);
        Debug.Print("Plus Tolerance: " + FormatDouble(plus * dbl));
        Debug.Print("Minus Tolerance: " + FormatDouble(minus * dbl));
        break;
    }
}

private void DistanceBetweenData(DimXpertDistanceBetweenDimTol annotation)
{
    DimXpertFeature feature = default(DimXpertFeature);
    swDimXpertDistanceFosUsage_e featureFosUsage = 0;
    double I = 0;
    double J = 0;
    double K = 0;
    bool boolstatus = false;

    feature = null;
    // the origin and tolerance feature aint with their feature of size usage (min, max, center)
    boolstatus = annotation.GetOriginFeature(ref feature, ref featureFosUsage);

    Debug.Print("");
    Debug.Print("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage));
    boolstatus = annotation.GetFeature(ref feature, ref featureFosUsage);
    Debug.Print("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage));

    // The direction vector

    boolstatus = annotation.GetDirectionVector(ref I, ref J, ref K);
    Debug.Print("");
    Debug.Print("Direction Vector: " + FormatVector(I, J, K));
}

private string annotationTypeNameFromObject(DimXpertAnnotation anno)
{
    return annotationTypeNameFromTypeNumber(anno.Type);
}

private string annotationTypeNameFromTypeNumber(swDimXpertAnnotationType_e annoTypeIndex)
{
    string functionReturnValue = null;
    switch (annoTypeIndex)
    {
        case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween:
            functionReturnValue = "DistanceBetween Dim";
            break;
        case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween:
            functionReturnValue = "AngleBetween Dim";
            break;
        default:
            functionReturnValue = "Unknown";
            break;
    }
    return functionReturnValue;
}

```

```

private string DisplayEntity(DimXpertAnnotation annotation)
{
    string str = null;
    object dispEnt = null;
    Annotation swAnnot = default(Annotation);

    dispEnt = annotation.GetDisplayEntity();

    str = null;
    if ((dispEnt != null))
    {
        if (dispEnt is Annotation)
        {
            swAnnot = (Annotation)dispEnt;
            str = swAnnot.GetName();
        }
    }
    return str;
}

private string FosUsage(swDimXpertDistanceFosUsage_e value)
{
    string str = null;
    switch (value)
    {
        case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_Center:
            str = "Center";
            break;
        case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MaximumSide:
            str = "Max";
            break;
        case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MinimumSide:
            str = "Min";
            break;
        default:
            str = "N/A";
            break;
    }
    return str;
}

private string FormatVector(double I, double J, double K)
{
    string str = null;
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K);
    return str;
}

private string FormatDouble(double value)
{
    string str = null;
    str = value.ToString();
    return str;
}

private string RadiansToDegrees(double value)
{
    string str = null;
    str = (value * 57.2957795130823).ToString();
    return str;
}
public SldWorks swApp;
}
}

```

Get and Set Location Dimension Example (VBA)

This example shows how to get and set DimXpert distance-between or angle-between dimensions.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\dimxpert\bracket_ads_plusminus.sldprt.
' 2. Multi-select two faces that:
'   a. share a common edge to get the angle-between dimension.
'   b. do not share a common edge to get the distance-between dimension.
' 3. Open an Immediate window.
' 4. Ensure that the latest SOLIDWORKS DimXpert type library is loaded:
'   a. Select Tools > References.
'   b. Click Browse.
'   c. Find and select install_dir\swdimxpert.tlb.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. Observe the DistanceBetween1 or AngleBetween1 annotation on the
'    DimXpertManager tab.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----

Option Explicit
Dim dimXpertPart As dimXpertPart

Sub Main()
    Dim swapp As SldWorks.SldWorks
    Set swapp = Application.SldWorks
    Dim swModelDoc As SldWorks.ModelDoc2
    Set swModelDoc = swapp.ActiveDoc
    Dim annoType As Long

    If swModelDoc Is Nothing Then
        Exit Sub
    End If

    Dim dimXpertMgr As SldWorks.DimXpertManager
    Set dimXpertMgr = swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name, True)
    Debug.Print "Model: " & swapp.IActiveDoc2.GetPathName

    Dim dimXpertPartObj As dimXpertPart
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart
    Set dimXpertPart = dimXpertPartObj

    Dim dimOption As DimXpertDimensionOption
    Set dimOption = dimXpertPart.GetDimOption

    ' Specify a dimension along the normal to the face
    dimOption.DimensionPositionOption = 0

    ' Insert the dimension
    dimXpertPart.InsertLocationDimension dimOption

    Dim vAnnotations As Variant
    vAnnotations = dimXpertPart.GetAnnotations()
    Debug.Print "-----"
    Debug.Print "Annotations..."
    Debug.Print "-----"

    Dim annotationTemp As DimXpertAnnotation
    Dim annotationIndex As Long
    For annotationIndex = 0 To UBound(vAnnotations)
        Set annotationTemp = vAnnotations(annotationIndex)
        Call GeneralInfo(annotationTemp)
        annoType = annotationTemp.Type
        Call DimensionToleranceData(annotationTemp)
    Next
End Sub

Private Sub GeneralInfo(annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Object
    Dim modelFeature As SldWorks.feature

    Debug.Print ""
    Debug.Print ("Name: " + annotation.Name)
    annoType = annotation.TypeNameFromObject(annotation)
    Debug.Print ("Type: " + annoType)
    Debug.Print ("Display Entity: " + DisplayEntity(annotation))

    Set modelObj = annotation.GetModelFeature
    Set modelFeature = modelObj

```

```

If Not (modelFeature Is Nothing) Then
    Debug.Print ("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2() + ")")
End If
End Sub

Private Sub DimensionToleranceData(annotation As DimXpertDimensionTolerance)
    Dim isAngleType As Boolean
    Dim annoType As Long
    Dim upper As Double, lower As Double
    Dim plus As Double, minus As Double
    Dim boolstatus As Boolean

    annoType = annotation.Type
    isAngleType = False

    Debug.Print ""
    Debug.Print ("Dimension Tolerance Compartment")

    If annoType = swDimXpertDimTol_DistanceBetween Then
        Dim distancebetween As DimXpertDistanceBetweenDimTol
        Set distancebetween = annotation
        Call DistanceBetweenData(distancebetween)
    ElseIf annoType = swDimXpertDimTol_AngleBetween Then
        isAngleType = True
        Dim angleBetween As IDimXpertAngleBetweenDimTol
        Set angleBetween = annotation

        ' the origin and tolerance feature
        Debug.Print ("Origin Feature: " + angleBetween.OriginFeature.Name)

        ' is supplement angle
        Debug.Print ("Supplement Angle: " + IIf(angleBetween.Supplement, "True", "False"))
    End If

    ' conversion for radians to degrees when dimension type is angle
    Dim dbl As Double
    If isAngleType Then
        dbl = 57.2957795130823
    Else
        dbl = 1#
    End If

    ' the nominal, and upper and lower limits of size of the dimension
    Debug.Print ("")
    Debug.Print ("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue * dbl))
    boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
    Debug.Print ("Get Upper Limit: " + FormatDouble(upper * dbl))
    Debug.Print ("Get Lower Limit: " + FormatDouble(lower * dbl))

    ' the upper and lower tolerance value by type
    Select Case annotation.DimensionType
    Case swDimXpertDimTolType_BlockTolerance
        Debug.Print ("Dimension Type: Block Tolerance")

        ' block tolerance
    Case swDimXpertDimTolType_BlockToleranceNoNominal
        Dim blockTols As DimXpertBlockTolerances
        Set blockTols = dimXpertPart.GetBlockTolerances
        Select Case blockTols.Type
        Case swDimXpertBlockToleranceType_ASMEInch
            Debug.Print ("Dimension Type: Block Tolerance No Nominal")
            If isAngleType Then
                Debug.Print ("Angular Block Tolerance")
            Else
                Debug.Print ("Block Tolerance Decimal Places: " + Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
            End If
        Case swDimXpertBlockToleranceType_ISO2768
            Debug.Print ("Dimension Type: General Tolerance")
        End Select
    Case swDimXpertDimTolType_ISOLimitsAndFits
        Debug.Print ("Dimension Type: Limits and Fits")

        ' limits and fits tolerance
    Case swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
        Debug.Print ("Dimension Type: Limits and Fits No Nominal")
        Debug.Print ("Limits and Fits: " + annotation.LimitsAndFitsCode)

        ' limit dimension
    Case swDimXpertDimTolType_LimitDimension
        Debug.Print ("Dimension Type: Limit Dimension")
        boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
        Debug.Print ("Get Upper Limit: " + FormatDouble(upper * dbl))
        Debug.Print ("Get Lower Limit: " + FormatDouble(lower * dbl))
    Case swDimXpertDimTolType_MAXTolerance
        Debug.Print ("Dimension Type: MAXTolerance")
    Case swDimXpertDimTolType_MINTolerance
        Debug.Print ("Dimension Type: MINTolerance")
    Case swDimXpertDimTolType_NoTolerance
        Debug.Print ("Dimension Type: NoTolerance")
    End Select
End Sub

```

```

    Case swDimXpertDimTolType_PlusMinusDimension
        Debug.Print ("Dimension Type: Plus Minus Dimension")
    ' plus and minus dimension
    Case swDimXpertDimTolType_PlusMinusNoNominal
        Debug.Print ("Dimension Type: Plus Minus No Nominal")
        boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
        Debug.Print ("Plus Tolerance: " + FormatDouble(plus * dbl))
        Debug.Print ("Minus Tolerance: " + FormatDouble(minus * dbl))
    End Select

End Sub

Private Sub DistanceBetweenData(annotation As DimXpertDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    ' the origin and tolerance feature along with their feature of size usage (min, max, center)
    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
    Debug.Print ("")
    Debug.Print ("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    Debug.Print ("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    ' The direction vector
    boolstatus = annotation.GetDirectionVector(I, J, K)
    Debug.Print ("")
    Debug.Print ("Direction Vector: " + FormatVector(I, J, K))
End Sub

Private Function annotationTypeNameFromObject(anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function

Private Function annotationTypeNameFromTypeNumber(annoTypeIndex As Long) As String
    Select Case annoTypeIndex
        Case swDimXpertDimTol_DistanceBetween
            annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
    End Select
End Function

Private Function DisplayEntity(annotation As DimXpertAnnotation) As String
    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As SldWorks.annotation
    Set dispEnt = annotation.GetDisplayEntity
    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is SldWorks.annotation Then
            Set swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If
    DisplayEntity = str
End Function

Private Function FosUsage(value As Long) As String
    Dim str As String
    Select Case value
        Case swDimXpertDistanceFosUsage_Center
            str = "Center"
        Case swDimXpertDistanceFosUsage_MaximumSide
            str = "Max"
        Case swDimXpertDistanceFosUsage_MinimumSide
            str = "Min"
        Case Else
            str = "N/A"
    End Select
    FosUsage = str
End Function

Private Function FormatVector(I As Double, J As Double, K As Double) As String
    Dim str As String
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)
    FormatVector = str
End Function

Private Function FormatDouble(value As Double) As String
    Dim str
    str = Format(value, "##,##0.0000")
    FormatDouble = str
End Function

Private Function RadiansToDegrees(value As Double) As String

```

```
Dim str
str = Format((value * 57.2957795130823), "##,##0.00")
RadiansToDegrees = str
End Function
```